ESGI Lyon
5ESGI AL + IABD, année 2026-2027

# Clusterisation de conteneurs

Rapport des travaux dirigés

Docker, Docker Compose, Docker Swarm, Traefik, Portainer,
Dozzle, Authelia, Outline

Ahmed Yahya LETAIEF

Novembre 2026

# Sommaire

**Présentation** **4**

**TD 1 : Dockerisation de My Favorite Places** **5**

1. Test initial avec Bruno . . . 5
1.1 Lancer une base PostgreSQL . . . 5
1.2 Lancer l'API en mode développement . . . 5
1.3 Tests avec la collection Bruno . . . 5
2. Dockerisation du serveur . . . 6
2.1 Le Dockerfile du serveur . . . 6
2.2 Adapter la connexion à la base . . . 6
2.3 Lancer la base et le serveur . . . 6
2.4 Test avec Bruno . . . 8
3. Dockerisation du client . . . 8
3.1 Le Dockerfile du client . . . 8
3.2 Adapter l'adresse de l'API . . . 8
3.3 Lancer le client . . . 8
3.4 Test dans le navigateur . . . 9
4. Orchestration avec Docker Compose . . . 9
4.1 Le fichier compose.yml . . . 9
4.2 Lancer l'application . . . 10
4.3 Test . . . 10
4.4 Simplification de la connexion à la base . . . 10
5. Communication entre les services . . . 11
5.1 Entre la base et le serveur . . . 11
5.2 Entre le client et le serveur . . . 11
6. Démarrage, redémarrage et images légères . . . 11
6.1 Démarrer le serveur quand la base est prête . . . 11
6.2 Redémarrage automatique . . . 11
6.3 Alléger les images . . . 11
6.3.a Serveur . . . 12
6.3.b Client . . . 12
6.3.c Résultat . . . 12

**TD 2 : Cluster Docker Swarm** **13**

Prérequis : images publiées sur un registry . . . 13
0.1 Publier les images avec GitHub Actions . . . 13
0.2 Vérifier les images sur GHCR . . . 13

Exercice 2 : Création du cluster Docker Swarm . . . 13
2.1 Lancer les 4 machines . . . 13
2.2 Initialiser le cluster . . . 14
2.3 Faire rejoindre les nœuds . . . 14
2.4 Lister les nœuds du cluster . . . 14
Exercice 3 : Tests du cluster . . . 15
3.1 Créer une stack hello-world . . . 15
3.2 Copier la stack dans le manager . . . 15
3.3 Déployer la stack et observer le cluster . . . 15
3.3.a Relancer le cluster après un redémarrage du PC . . . 15
3.3.b Déployer la stack hello . . . 16
3.4 Faire varier la clause deploy . . . 17
3.4.a Sur le manager uniquement . . . 17
3.4.b Sur tous les nœuds (mode global) . . . 18

**TD 3 : Traefik et Portainer 20**

Exercice 1 : Déployer Traefik . . . 20
1.1 Relancer le cluster . . . 20
1.2 Déployer Traefik sur le manager . . . 20
1.3 Accéder à Traefik depuis le navigateur . . . 21
Exercice 2 : Déployer une autre application . . . 22
2.1 Tester l'application avec Docker Compose . . . 22
2.2 Déployer l'application sur le cluster Swarm . . . 23
2.3 Publier le vote et les résultats avec Traefik . . . 23
Exercice 3 : Déployer Portainer . . . 24
3.1 Déployer Portainer sur le cluster. . . . 24
3.2 Publier Portainer avec Traefik . . . 25
3.3 Ouvrir Portainer dans le navigateur . . . 25
Exercice 4 : Redéployer depuis Portainer . . . 26
4.1 Supprimer la stack depuis le manager . . . 26
4.2 Republier la stack depuis Portainer . . . 26
4.3 Vérifier l'accès depuis le navigateur . . . 27
4.4 Déployer l'API de MFP depuis Portainer . . . 27
4.4.a Création de la stack . . . 27
4.4.b Test avec Bruno . . . 28

**TD 4 : Secrets et configs Docker Swarm 29**

Prérequis : relancer le cluster et passer en HTTPS . . . 29
0.1 Relancer le cluster . . . 29
0.2 Créer le certificat . . . 30
0.3 Le certificat dans le cluster : un secret et des configs . . . 30
0.4 Traefik en HTTPS . . . 31

0.5 Relancer Portainer et ajouter les adresses . . . 32
Exercice 1 : Déployer des logiciels . . . 32
1.1 Dozzle avec un utilisateur simple . . . 32
1.1.a Le fichier users.yml . . . 32
1.1.b Le secret, créé dans Portainer . . . 32
1.1.c La stack . . . 33
1.1.d Test dans le navigateur . . . 33
1.2 Authelia et connexion OpenID Connect . . . 33
1.2.a Les clés d'Authelia . . . 34
1.2.b Le compte dans un secret. . . . 34
1.2.c Le client OIDC de Dozzle . . . 34
1.2.d La configuration dans une config . . . 34
1.2.e Le déploiement . . . 35
1.2.f Brancher Dozzle sur Authelia . . . 36
1.2.g Rôles manquants et mise à jour de la config . . . 36
1.2.h Résultat . . . 37
1.3 Bonus : Outline . . . 38
1.3.a Les secrets d'Outline . . . 38
1.3.b Un nouveau client dans Authelia . . . 38
1.3.c La stack . . . 39
1.3.d Résultat . . . 39
Bilan. . . . 40

# Présentation

**Dépôt :** `https://github.com/DARK-SHAD0W/Clusterisation-Conteneurs`

- `mfp/` : l'application My Favorite Places (client React, API Node.js/Express, base PostgreSQL) et le cluster Docker Swarm (`mfp/swarm/`) ;
- `voting-app/` : l'application de vote du TD 3.

**Environnement de travail :**

| Outil | Version | Remarque |
|---|---|---|
| Docker Engine | 29.7.2 | dans WSL 2 (Ubuntu 24.04), sans Docker Desktop |
| Docker Compose | v5.5.0 | plugin `docker compose` |
| Docker des nœuds du cluster | 29.8.2 | image `docker:dind` |
| Node.js / Yarn | 22.23.2 / 1.22.22 | dans WSL |
| Bruno | 4.2.1 | client API, sous Windows |

Les commandes du cluster se lancent depuis le dossier `mfp/swarm/`, qui contient son `compose.yml`.

# TD 1 : Dockerisation de My Favorite Places

My Favorite Places se compose d'un client React, d'une API Node.js/Express et d'une base PostgreSQL. Dans ce TD, je les ai mis dans des conteneurs, puis orchestrés avec Docker Compose.

## 1. Test initial avec Bruno

### 1.1 Lancer une base PostgreSQL

Le serveur ne démarre pas sans base : j'ai lancé PostgreSQL avec le mot de passe attendu par `server/src/datasource.ts`.

**Commandes**

```
docker run -d --name mfp-db \
  -e POSTGRES_PASSWORD=supersecret \
  -p 5432:5432 \
  postgres:17
docker ps
```

### 1.2 Lancer l'API en mode développement

**Commandes**

```
cd server
yarn install
yarn dev
```

### 1.3 Tests avec la collection Bruno

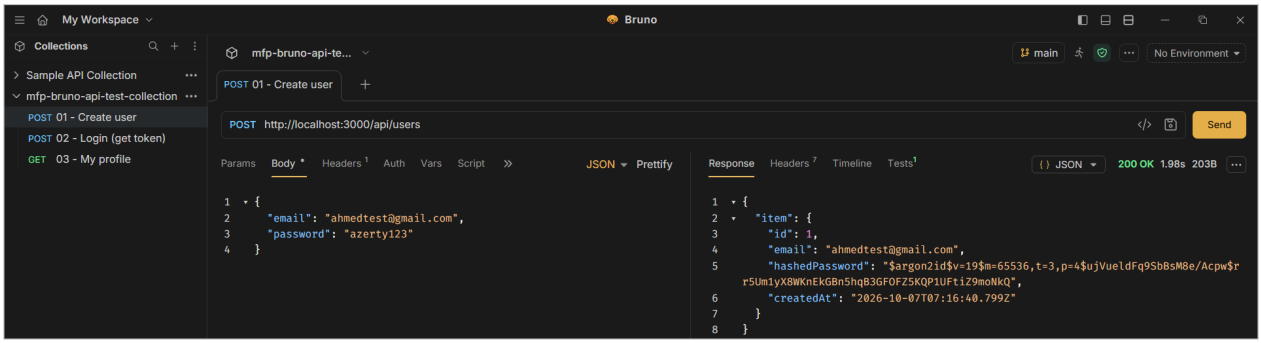



**Figure 1.1.** L'utilisateur est créé et son mot de passe est stocké haché avec Argon2.

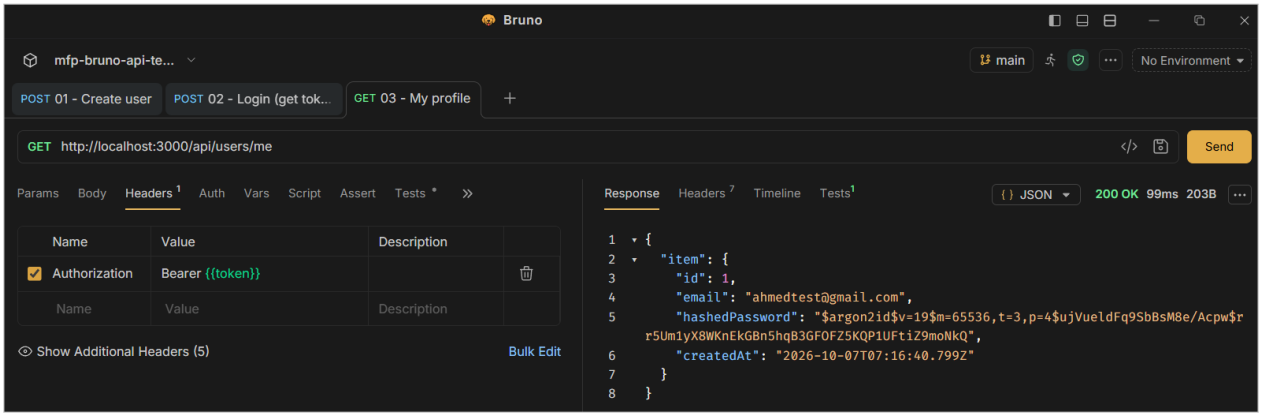



**Figure 1.2.** Avec le token obtenu à la connexion, `/api/users/me` renvoie l'utilisateur connecté.

# 2. Dockerisation du serveur

## 2.1 Le Dockerfile du serveur

**server/Dockerfile**

```
FROM node:24-alpine3.24
WORKDIR /app
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile
COPY tsconfig.json ./
COPY src src
RUN yarn build
EXPOSE 3000
CMD ["yarn", "start"]
```

J'installe les dépendances avant de copier le code, pour que Docker garde cette étape en cache tant que `yarn.lock` ne change pas. Les captures ont été prises avec l'image `node:22`, que j'ai remplacée ensuite par `node:24-alpine3.24`.

## 2.2 Adapter la connexion à la base

Dans un conteneur, `localhost` désigne le conteneur lui-même : le serveur lit donc l'adresse de la base dans des variables d'environnement.

**server/src/datasource.ts**

```
host: process.env.DB_HOST || "localhost",
username: process.env.DB_USER || "postgres",
password: process.env.DB_PASSWORD || "supersecret",
database: process.env.DB_NAME || "postgres",
```

## 2.3 Lancer la base et le serveur

Le serveur ne peut appeler la base par son nom, `mfp-db`, que si les deux conteneurs partagent un réseau.

**Commandes**

```
docker network create mfp-net
docker run -d --name mfp-db --network mfp-net \
```

```
  -e POSTGRES_PASSWORD=supersecret \
  postgres:17
docker build -t mfp-server ./server
docker run -d --name mfp-server --network mfp-net \
  -e DB_HOST=mfp-db \
  -p 3000:3000 \
  mfp-server
```

**Problème rencontré :** `address already in use` : le `yarn dev` de la partie 1 occupait le port 3000.
**Solution :** J'ai arrêté `yarn dev`, supprimé le conteneur raté et relancé le serveur.

**Commandes**

```
docker rm -f mfp-server
docker run -d --name mfp-server --network mfp-net \
  -e DB_HOST=mfp-db \
  -p 3000:3000 \
  mfp-server
docker ps
docker logs mfp-server
```

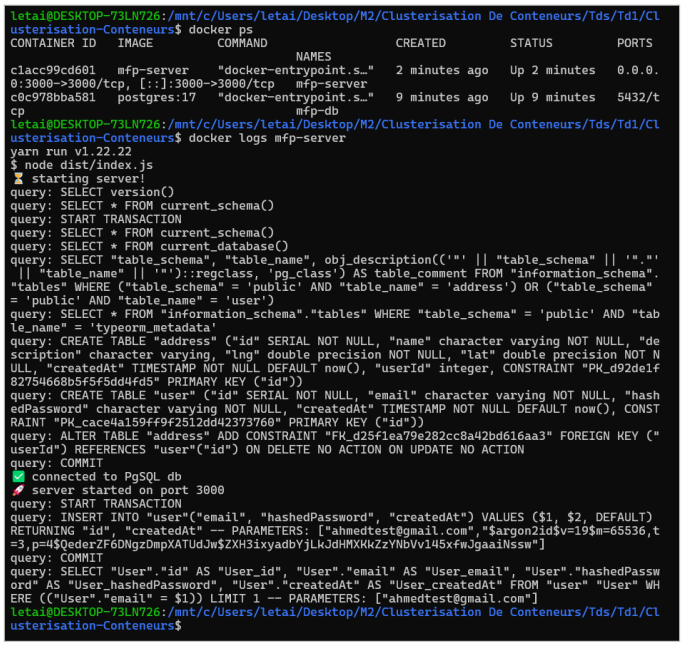



**Figure 1.3.** Le serveur conteneurisé crée les tables `user` et `address` dans `mfp-db`, puis démarre.

### 2.4 Test avec Bruno

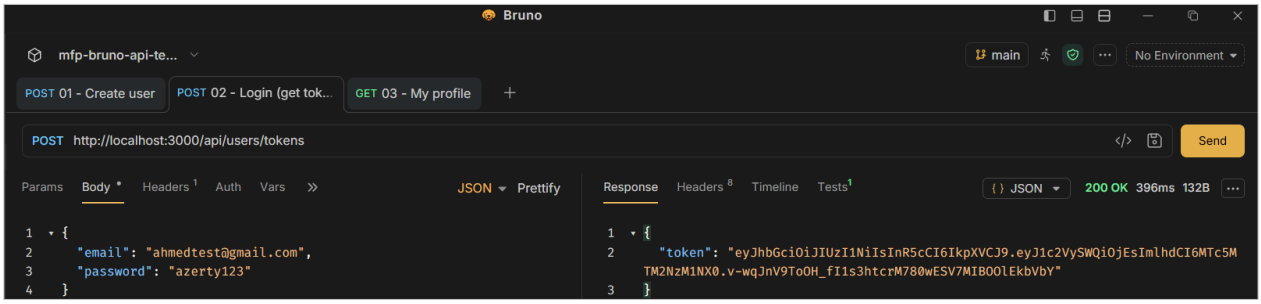



**Figure 1.4.** Après avoir recréé l'utilisateur dans la base neuve, le serveur dans Docker renvoie bien un token.

## 3. Dockerisation du client

### 3.1 Le Dockerfile du client

**client/Dockerfile**

```
FROM node:24-alpine3.24
WORKDIR /app
COPY package*.json yarn.lock ./
RUN yarn install --frozen-lockfile
# ...
EXPOSE 5173
CMD ["yarn", "dev", "--host"]
```

Sans `--host`, Vite refuse les connexions venant de l'extérieur du conteneur.

### 3.2 Adapter l'adresse de l'API

Vite relaie les appels `/api` vers `localhost:3000`, qui dans ce conteneur n'est plus le serveur : l'adresse passe donc par une variable.

**client/vite.config.ts**

```
target: process.env.API_URL || "http://localhost:3000",
```

### 3.3 Lancer le client

**Commandes**

```
docker build -t mfp-client ./client
docker run -d --name mfp-client --network mfp-net \
  -e API_URL=http://mfp-server:3000 \
  -p 5173:5173 \
  mfp-client
docker ps
docker logs mfp-client
```

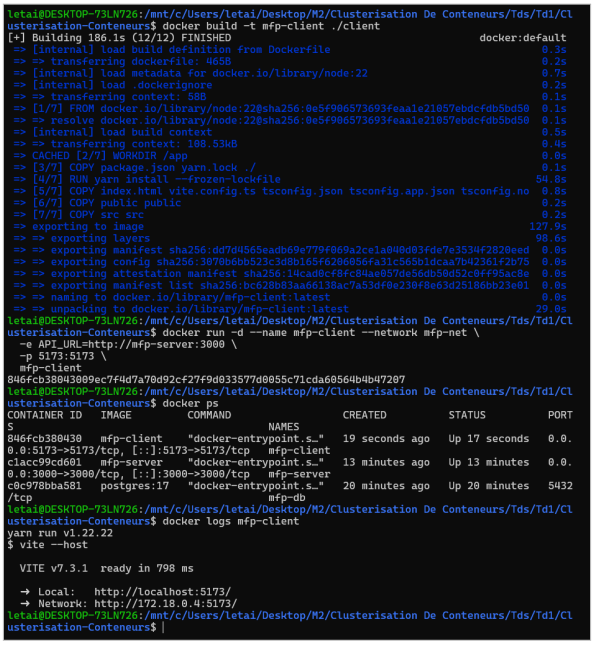



**Figure 1.5.** Les trois conteneurs tournent et Vite écoute sur le port 5173.

### 3.4 Test dans le navigateur

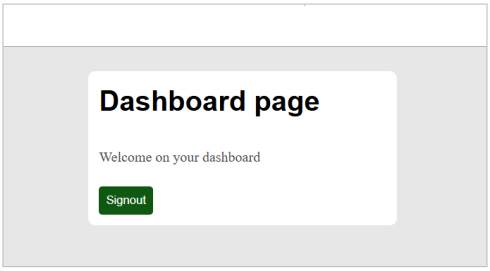



**Figure 1.6.** La connexion fonctionne depuis le client conteneurisé : les appels `/api` atteignent le serveur.

## 4. Orchestration avec Docker Compose

### 4.1 Le fichier compose.yml

Compose crée lui-même le réseau, sur lequel chaque service est joignable par son nom.

**compose.yml**

```yaml
services:
  db:
    # ...
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 2s
      # ...
  server:
    # ...
    depends_on:
      db:
        condition: service_healthy
```

## 4.2 Lancer l'application

Les conteneurs lancés à la main occupaient encore les ports 3000 et 5173 : je les ai supprimés avant de lancer Compose.

**Commandes**

```
docker rm -f mfp-client mfp-server mfp-db
docker network rm mfp-net
docker compose up -d --build
docker compose ps
docker compose logs server
```

**Problème rencontré :** Le serveur s'arrêtait : `depends_on` seul n'attend pas que PostgreSQL soit prêt.
**Solution :** J'ai ajouté le `healthcheck` et `condition: service_healthy` montrés en 4.1.

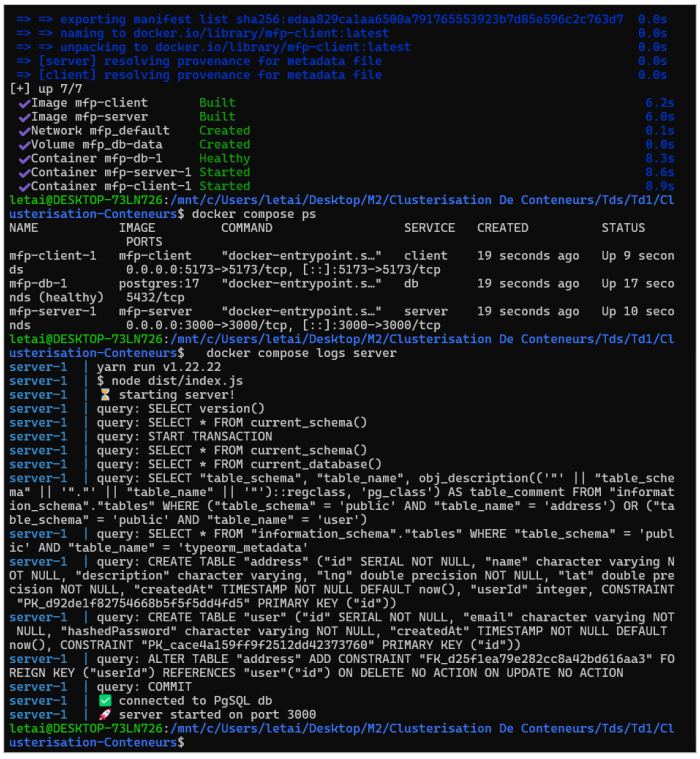



**Figure 1.7.** `mfp-db-1` devient `Healthy` avant le démarrage du serveur, qui se connecte du premier coup.

## 4.3 Test

Le volume étant neuf, j'ai recréé l'utilisateur ; la connexion au client fonctionne ensuite comme en 3.4. L'arrêt avec `down` ne supprime pas le volume `db-data`.

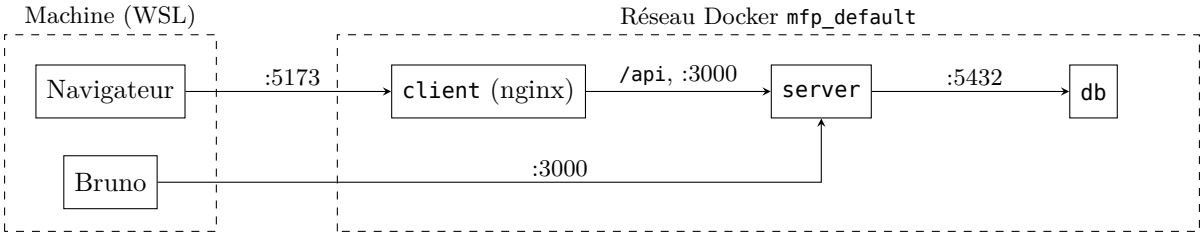
**Commandes**

```
docker compose down
```

## 4.4 Simplification de la connexion à la base

Avec Compose, la base s'appelle toujours `db` : dans `datasource.ts`, j'ai remplacé les variables d'environnement par cette valeur fixe, mais le serveur ne trouve plus la base hors de Compose.

# 5. Communication entre les services




## 5.1 Entre la base et le serveur

Le serveur appelle la base par son nom de service, `db`, que Docker résout sur le réseau `mfp_default`. Le port 5432 n'est publié que pour consulter la base depuis ma machine.

## 5.2 Entre le client et le serveur

Le code React s'exécute dans le navigateur, hors du réseau Docker : il appelle `/api` à l'adresse du site, et le conteneur `client` relaie ces appels vers `server`, d'abord avec Vite puis avec nginx.

# 6. Démarrage, redémarrage et images légères

## 6.1 Démarrer le serveur quand la base est prête

C'était déjà réglé en 4.2 avec le `healthcheck` de `db` et `condition: service_healthy`.

## 6.2 Redémarrage automatique

Avec `unless-stopped`, Docker relance un service qui plante, mais pas un service que j'arrête à la main.

**compose.yml (chaque service)**

```
restart: unless-stopped
```

**Commandes**

```
docker compose exec server pkill -f "node dist/index.js"
docker inspect mfp-server-1 --format 'status={{.State.Status}} restarts={{.RestartCount}}'
# status=running restarts=1

docker compose stop server
docker inspect mfp-server-1 --format 'status={{.State.Status}} restarts={{.RestartCount}}'
# status=exited restarts=1   (arrêt manuel : pas de redémarrage)

docker compose start server
```

## 6.3 Alléger les images

Chaque Dockerfile passe à trois étapes. Seule la dernière devient l'image finale : les outils de construction et le code source restent dans les étapes précédentes.

### 6.3.a Serveur

**server/Dockerfile**

```
FROM node:24-alpine3.24 AS deps
# ...
FROM node:24-alpine3.24 AS builder
COPY --from=deps /app/node_modules ./node_modules
# ...
FROM node:24-alpine3.24 AS main
# ...
COPY --from=builder /app/dist ./dist
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile --production && yarn cache clean
# ...
CMD ["yarn", "start"]
```

### 6.3.b Client

Les étapes `deps` et `builder` sont celles du serveur. Une fois construit, le client n'est plus que des fichiers statiques, que nginx sert à la place de Vite ; `try_files` renvoie `index.html` pour les routes React comme `/dashboard`.

**client/Dockerfile**

```
# ...
FROM nginx:1.29-alpine AS main
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 5173
```

**client/nginx.conf**

```
server {
    listen 5173;
    # ...
    location / {
        try_files $uri $uri/ /index.html;
    }
    location /api/ {
        proxy_pass http://server:3000;
    }
}
```

### 6.3.c Résultat

**Commandes**

```
docker compose up -d --build
docker images
```

| Image | Avant | Après |
|---|---|---|
| `mfp-server` | 477 Mo | 337 Mo |
| `mfp-client` | 1,49 Go | 93 Mo |

# TD 2 : Cluster Docker Swarm

J'ai monté un cluster Docker Swarm local de 4 machines (1 manager et 3 nœuds), chacune étant un conteneur Docker-in-Docker.

## Prérequis : images publiées sur un registry

### 0.1 Publier les images avec GitHub Actions

Swarm ne sait pas construire d'images : j'ai donc écrit deux workflows qui les construisent à chaque push sur `main` et les publient sur GHCR.

**.github/workflows/build.server.yml**

```yaml
env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}/server
```

### 0.2 Vérifier les images sur GHCR

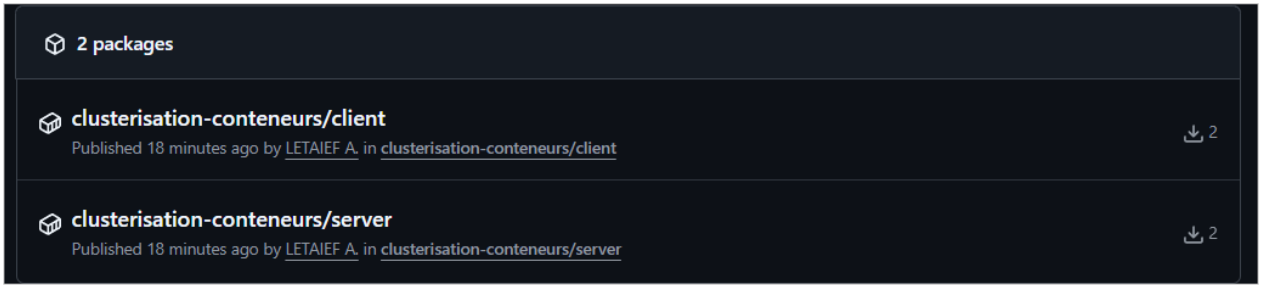



**Figure 2.1.** Les deux images sont publiques : les nœuds les téléchargent sans connexion.

## Exercice 2 : Création du cluster Docker Swarm

### 2.1 Lancer les 4 machines

Les nœuds `node1` à `node3` sont définis comme le manager, et `privileged: true` permet de faire tourner Docker dans un conteneur.

**swarm/compose.yml**

```yaml
services:
  manager:
    image: docker:dind
    hostname: manager
    privileged: true
  # ...
```

**Commandes**

```
cd swarm
docker compose up -d
docker ps
```

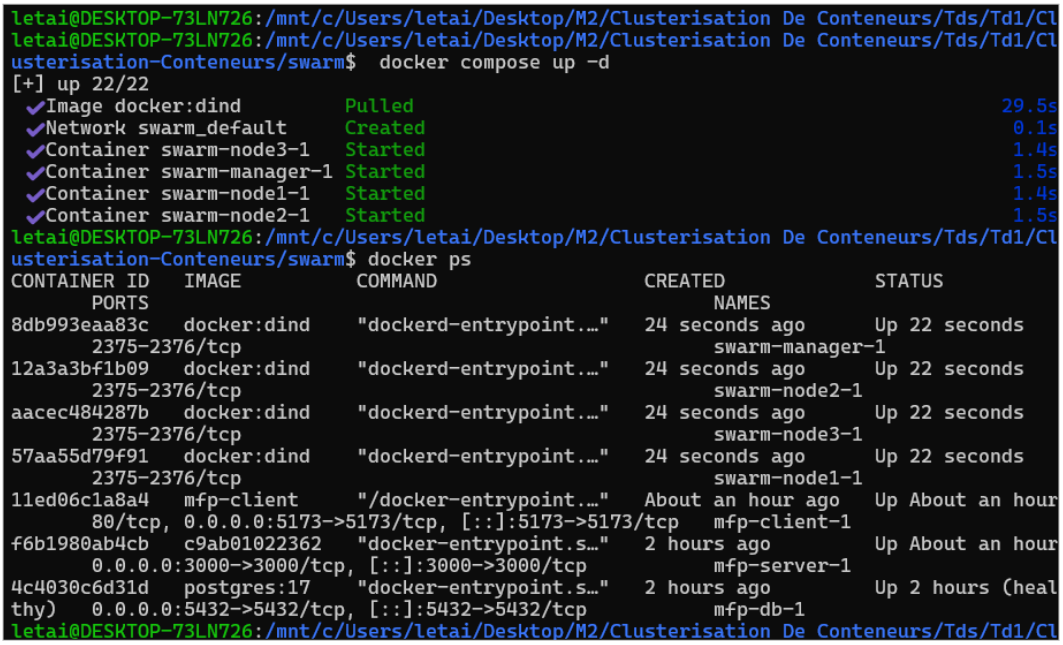



**Figure 2.2.** Les 4 machines du cluster tournent à côté des conteneurs du TD 1.

**Commandes**

```
docker compose exec manager docker ps
docker compose exec node1 docker ps
```

La liste est vide dans les deux cas : chaque machine a son propre Docker, séparé de celui du PC.

## 2.2 Initialiser le cluster

**Commandes**

```
docker compose exec manager docker swarm init
```

## 2.3 Faire rejoindre les nœuds

**Commandes**

```
TOKEN=$(docker compose exec manager docker swarm join-token -q worker)
docker compose exec node1 docker swarm join --token $TOKEN manager:2377
docker compose exec node2 docker swarm join --token $TOKEN manager:2377
docker compose exec node3 docker swarm join --token $TOKEN manager:2377
```

## 2.4 Lister les nœuds du cluster

**Commandes**

```
docker compose exec manager ash
docker node ls
exit
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusteris
ation-Conteneurs/swarm$  docker compose exec manager ash
/ #  docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
ut2j0ctz6094h5y236qwnbii5 *   manager    Ready     Active         Leader           29.8.2
gkqoimullj55ae70vubvjpq3t     node1      Ready     Active                          29.8.2
htlyy0s49t7h787sznf9syr0h     node2      Ready     Active                          29.8.2
tih4g88m7eipgra6a02nwvi17     node3      Ready     Active                          29.8.2
/ # exit
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusteris
ation-Conteneurs/swarm$
```

**Figure 2.3.** Les 4 nœuds sont `Ready`, avec le `manager` comme `Leader`.

# Exercice 3 : Tests du cluster

## 3.1 Créer une stack hello-world

**swarm/hello-world.compose.yml**

```yaml
# ...
deploy:
  replicas: 2
```

## 3.2 Copier la stack dans le manager

J'ai ajouté un volume anonyme sur `/home/manager` pour que la stack copiée survive à une recréation du conteneur.

**swarm/compose.yml**

```yaml
manager:
  # ...
  volumes:
    - /home/manager
```

**Commandes**

```
docker compose up -d manager
docker compose exec manager docker node ls
docker compose exec manager mkdir -p /home/manager
docker compose cp hello-world.compose.yml manager:/home/manager/
docker compose exec manager ls /home/manager
```

Après sa recréation, le manager voit toujours ses 4 nœuds `Ready`.

## 3.3 Déployer la stack et observer le cluster

### 3.3.a Relancer le cluster après un redémarrage du PC

**Commandes**

```
docker compose start
docker compose exec manager docker node ls
```

**Problème rencontré :** Swarm retient l'IP du manager, qui avait changé : les nœuds restaient `Down`.
**Solution :** J'ai fait quitter le cluster à toutes les machines, puis je l'ai recréé.

**Commandes**

```
docker compose exec node1 docker swarm leave --force
docker compose exec node2 docker swarm leave --force
docker compose exec node3 docker swarm leave --force
docker compose exec manager docker swarm leave --force

docker compose exec manager docker swarm init
TOKEN=$(docker compose exec manager docker swarm join-token -q worker)
docker compose exec node1 docker swarm join --token $TOKEN manager:2377
docker compose exec node2 docker swarm join --token $TOKEN manager:2377
docker compose exec node3 docker swarm join --token $TOKEN manager:2377

docker compose exec manager docker node ls
```

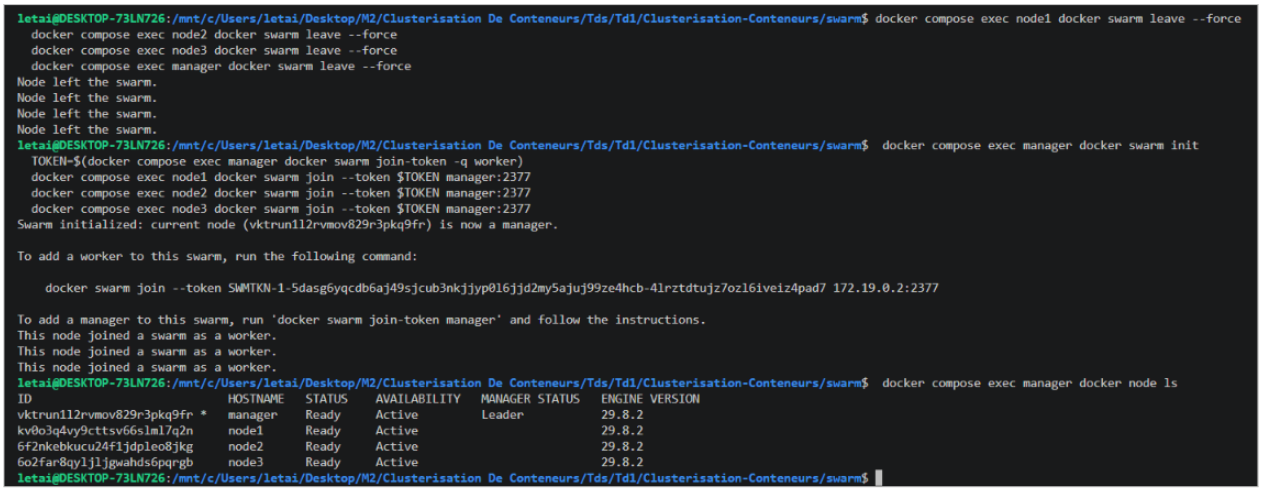



**Figure 2.4.** Le cluster recréé retrouve ses 4 nœuds `Ready`.

### 3.3.b Déployer la stack hello

**Commandes**

```
docker compose exec manager docker stack deploy -c /home/manager/hello-world.compose.yml
↪  hello
docker compose exec manager docker service ls
docker compose exec manager docker service ps hello_hello
```

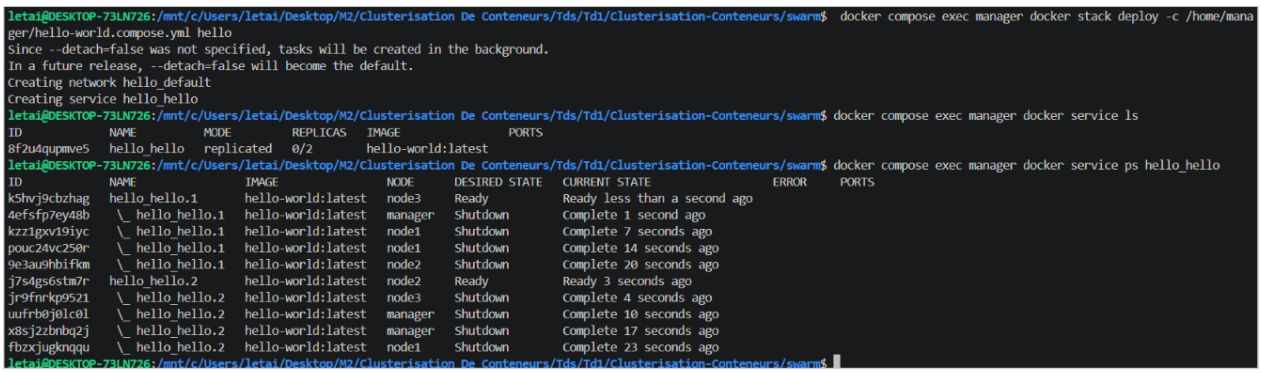



**Figure 2.5.** `hello-world` s'arrête dès son message affiché : le service reste à `0/2` et Swarm relance sans cesse les copies, sur tous les nœuds.

Pour comparer, un service nginx qui reste allumé affiche bien `3/3`.

**Commandes**

```
docker compose exec manager docker service create --name web --replicas 3 nginx:alpine
docker compose exec manager docker service ls
docker compose exec manager docker service rm web

docker compose exec manager docker ps -a
docker compose exec node1 docker ps -a
docker compose exec node2 docker ps -a
docker compose exec node3 docker ps -a
```

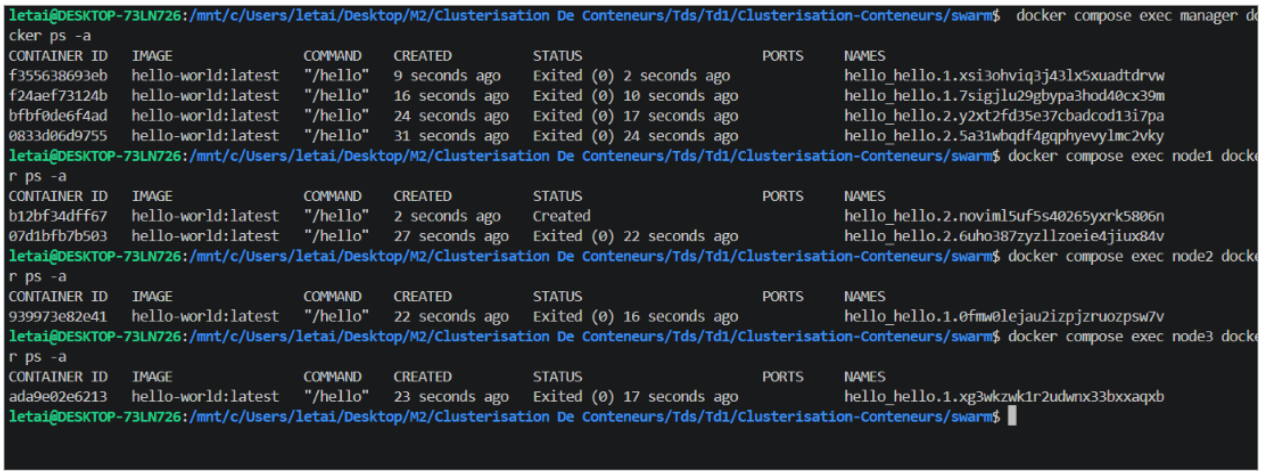



**Figure 2.6.** Swarm a réparti les copies sur les 4 machines, `manager` compris. Chaque machine ne voit que les siennes.

## 3.4 Faire varier la clause deploy

### 3.4.a Sur le manager uniquement

**swarm/hello-world-manager.compose.yml**

```yaml
# ...
deploy:
  replicas: 2
  placement:
    constraints:
      - node.role == manager
```

**Commandes**

```
docker compose exec manager docker stack rm hello
docker compose cp hello-world-manager.compose.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c
↪  /home/manager/hello-world-manager.compose.yml hello
docker compose exec manager docker service ps hello_hello
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
docker compose exec manager docker stack rm hello
Removing service hello_hello
Removing network hello_default
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
  docker compose cp hello-world-manager.compose.yml manager:/home/manager/
[+] copy 1/1
 ✓ swarm-manager-1 Copied hello-world-manager.compose.yml to swarm-manager-1:/home/manager/                    0.1s
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
 docker compose exec manager docker stack deploy -c /home/manager/hello-world-manager.compose.yml hello
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network hello_default
Creating service hello_hello
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
 docker compose exec manager docker service ps hello_hello
ID             NAME                IMAGE                NODE      DESIRED STATE   CURRENT STATE                ERROR
 PORTS
zfjhmrzuo4fz   hello_hello.1       hello-world:latest   manager   Ready           Ready less than a second ago

0z7vmg3rvvnu    \_ hello_hello.1   hello-world:latest   manager   Shutdown        Complete 1 second ago

0s8l81lgrk1a    \_ hello_hello.1   hello-world:latest   manager   Shutdown        Complete 7 seconds ago

qkg5g80puhls   hello_hello.2       hello-world:latest   manager   Ready           Ready less than a second ago

lod0t1rgd1ya    \_ hello_hello.2   hello-world:latest   manager   Shutdown        Complete 1 second ago

vq2ejxa6qsuo    \_ hello_hello.2   hello-world:latest   manager   Shutdown        Complete 7 seconds ago

letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

**Figure 2.7.** Toutes les copies tournent sur le `manager`, aucune sur les workers.

### 3.4.b Sur tous les nœuds (mode global)

**swarm/hello-world-global.compose.yml**

```yaml
# ...
deploy:
  mode: global
```

**Commandes**

```
docker compose exec manager docker stack rm hello
docker compose cp hello-world-global.compose.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c
↪  /home/manager/hello-world-global.compose.yml hello
docker compose exec manager docker service ls
docker compose exec manager docker service ps hello_hello
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager do
cker stack rm hello
Removing service hello_hello
Removing network hello_default
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose cp hello-world-
global.compose.yml manager:/home/manager/
[+] copy 1/1
 ✓ swarm-manager-1 Copied hello-world-global.compose.yml to swarm-manager-1:/home/manager/                                                    0.1s
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager doc
ker stack deploy -c /home/manager/hello-world-global.compose.yml hello
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network hello_default
Creating service hello_hello
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager do
cker service ls
ID             NAME          MODE      REPLICAS   IMAGE                PORTS
rno40lm6b9zl   hello_hello   global    0/0        hello-world:latest
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager do
cker service ps hello_hello
ID             NAME                                     IMAGE                NODE      DESIRED STATE   CURRENT STATE                ERROR     PORTS
p16kq27gcitt   hello_hello.6f2nkebkucu24f1jdpleo8jkg     hello-world:latest   node2     Ready           Ready less than a second ago
spzm3npdcc0n    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg hello-world:latest   node2     Shutdown        Complete 1 second ago
fbfe974s5a7d    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg hello-world:latest   node2     Shutdown        Complete 9 seconds ago
kfi1wj4o3psc    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg hello-world:latest   node2     Shutdown        Complete 15 seconds ago
ozlzlcgou9mc    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg hello-world:latest   node2     Shutdown        Complete 22 seconds ago
74fck10ko4cn   hello_hello.6o2far8qyljljgwahds6pqrgb     hello-world:latest   node3     Ready           Ready less than a second ago
wgp9hf0ivajv    \_ hello_hello.6o2far8qyljljgwahds6pqrgb hello-world:latest   node3     Shutdown        Complete 1 second ago
3274m4l4cf1d    \_ hello_hello.6o2far8qyljljgwahds6pqrgb hello-world:latest   node3     Shutdown        Complete 9 seconds ago
c8lp7t71y9x8    \_ hello_hello.6o2far8qyljljgwahds6pqrgb hello-world:latest   node3     Shutdown        Complete 15 seconds ago
68a30dpoer2i    \_ hello_hello.6o2far8qyljljgwahds6pqrgb hello-world:latest   node3     Shutdown        Complete 22 seconds ago
r7cz5gaxghs8   hello_hello.kv0o3q4vy9cttsv66slml7q2n     hello-world:latest   node1     Ready           Ready 3 seconds ago
uipommkun2bz    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n hello-world:latest   node1     Shutdown        Complete 4 seconds ago
iuw7w43mt34s    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n hello-world:latest   node1     Shutdown        Complete 11 seconds ago
md9jqa42g20l    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n hello-world:latest   node1     Shutdown        Complete 17 seconds ago
ntjsnf5aqd2t    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n hello-world:latest   node1     Shutdown        Complete 23 seconds ago
vrwzcp2e75xy   hello_hello.vktrun1l2rvmov829r3pkq9fr     hello-world:latest   manager   Ready           Ready less than a second ago
t9dwf3qketly    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr hello-world:latest   manager   Shutdown        Complete 2 seconds ago
ok3tbu0hd9zo    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr hello-world:latest   manager   Shutdown        Complete 9 seconds ago
rtsx0d3rbglq    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr hello-world:latest   manager   Shutdown        Complete 15 seconds ago
d5puswsuv948    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr hello-world:latest   manager   Shutdown        Complete 22 seconds ago
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

**Figure 2.8.** En mode `global`, Swarm lance une copie sur chacun des 4 nœuds.

# TD 3 : Traefik et Portainer

Dans ce TD, j'ai placé Traefik, un reverse proxy, devant mon cluster Swarm : chaque application répond à son propre nom de domaine au lieu d'un port. J'ai ensuite installé Portainer pour gérer le cluster depuis le navigateur.

## Exercice 1 : Déployer Traefik

### 1.1 Relancer le cluster

Traefik tourne dans le manager : j'ai publié le port 80 du manager pour que le navigateur du PC puisse le joindre.

**mfp/swarm/compose.yml**

```yaml
manager:
  # ...
  ports:
    - "80:80" # Traefik, reachable from the host browser
```

**Depuis mfp/swarm**

```
docker compose up -d
docker compose exec manager docker node ls
```

`up -d` n'a recréé que le manager, et les 4 nœuds sont restés `Ready`.

### 1.2 Déployer Traefik sur le manager

**mfp/swarm/traefik.compose.yml**

```yaml
whoami:
  # ...
  deploy:
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.whoami.rule=Host(`whoami.swarm.localhost`)"
      - "traefik.http.routers.whoami.entrypoints=web"
      - "traefik.http.services.whoami.loadbalancer.server.port=80"
```

La stack déclare le réseau `web` comme `external` : il doit exister avant le déploiement, en overlay pour couvrir tous les nœuds.

**Commandes**

```
docker compose exec manager docker network create --driver overlay web
docker compose exec manager docker network ls --filter name=web
docker compose exec manager docker stack rm hello
docker compose exec manager docker service rm web
docker compose cp traefik.compose.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/traefik.compose.yml traefik
docker compose exec manager docker service ls
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose cp traefik.compose.yml manager:/home/manager/
 swarm-manager-1 Copying traefik.compose.yml to swarm-manager-1:/home/manager/
 swarm-manager-1 Copied traefik.compose.yml to swarm-manager-1:/home/manager/
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager docker stack deploy -c /home/manage
r/traefik.compose.yml traefik
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating service traefik_traefik
Creating service traefik_whoami
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager docker service ls
ID             NAME              MODE         REPLICAS   IMAGE                 PORTS
34nw81r3tlo8   traefik_traefik   replicated   1/1        traefik:v3.6          *:80->80/tcp
mbp8wvpskz9n   traefik_whoami    replicated   1/1        traefik/whoami:latest
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

**Figure 3.1.** Seul Traefik publie un port (80) : whoami n'est joignable qu'à travers lui.

## 1.3 Accéder à Traefik depuis le navigateur

Aucun DNS ne connaît ces noms : je les ai déclarés dans le fichier hosts de Windows, ouvert en administrateur.

**C:\Windows\System32\drivers\etc\hosts**

```
127.0.0.1   traefik.swarm.localhost
127.0.0.1   whoami.swarm.localhost
```

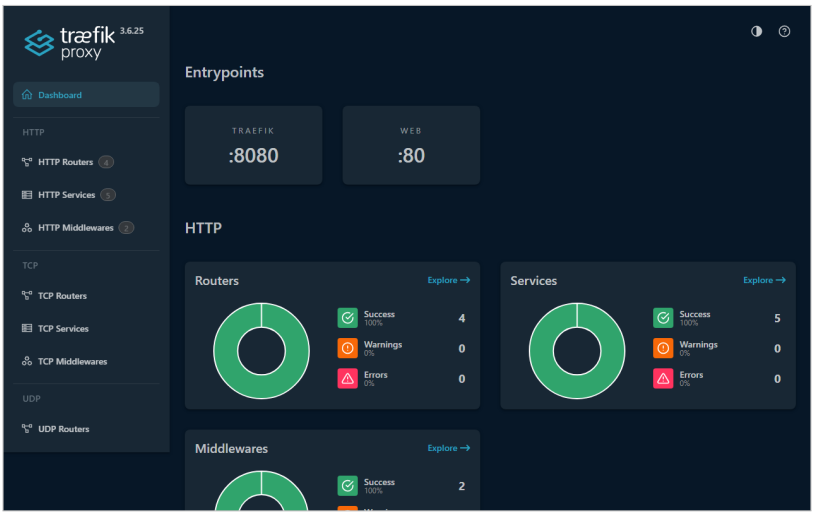



**Figure 3.2.** Traefik a découvert les routeurs tout seul grâce aux labels, sans aucune erreur.

```
Hostname: e96fb8cd89f8
IP: 127.0.0.1
IP: ::1
IP: 10.0.2.6
IP: 172.18.0.3
RemoteAddr: 10.0.2.3:34768
GET / HTTP/1.1
Host: whoami.swarm.localhost
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
HeadlessChrome/156.0.0.0 Safari/537.36 Edg/156.0.0.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/
signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: fr,fr-FR;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Sec-Ch-Ua: "Not:A-Brand";v="8", "Chromium";v="156", "Microsoft Edge";v="156"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Upgrade-Insecure-Requests: 1
X-Forwarded-For: 10.0.0.2
X-Forwarded-Host: whoami.swarm.localhost
X-Forwarded-Port: 80
X-Forwarded-Proto: http
X-Forwarded-Server: 62ea62114d18
X-Real-Ip: 10.0.0.2
```

**Figure 3.3.** Traefik a routé la requête grâce à l'en-tête `Host`, et ses en-têtes `X-Forwarded-*` prouvent qu'elle est passée par lui.

# Exercice 2 : Déployer une autre application

## 2.1 Tester l'application avec Docker Compose

**Depuis voting-app**

```
docker compose up -d
docker compose ps --format 'table {{.Name}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
curl -X POST -d vote=a http://localhost:8080/   # Cats, 3 fois
curl -X POST -d vote=b http://localhost:8080/   # Dogs, 2 fois
```

**Problème rencontré :** Docker Hub a répondu `429 Too Many Requests` : il limite les téléchargements sans compte.

**Solution :** Je me suis connecté avec `docker login`, puis j'ai relancé `docker compose up -d`.

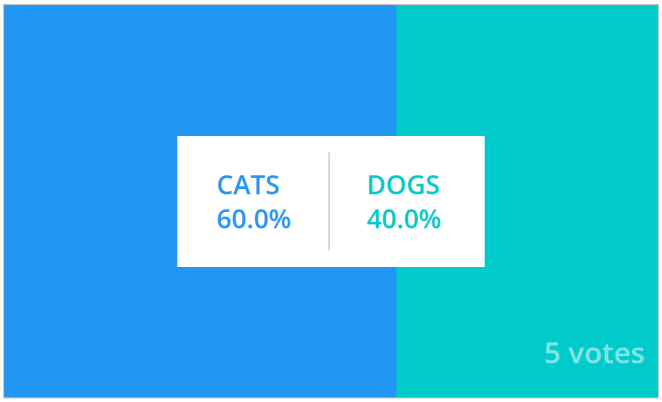



**Figure 3.4.** Les 5 votes ont traversé redis, le worker et la base jusqu'au site des résultats.

**Commandes**

```
docker compose down
```

## 2.2 Déployer l'application sur le cluster Swarm

Swarm ne construit pas d'images : la stack utilise des images publiées, et `--with-registry-auth` transmet ma connexion Docker Hub aux nœuds qui les téléchargent.

**Depuis mfp/swarm**

```
docker compose exec manager docker login
docker compose cp ../../voting-app/docker-stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy --with-registry-auth -c
↪  /home/manager/docker-stack.yml voting
docker compose exec manager docker stack services voting
```

Les 5 services étaient au complet après une minute, mais les ports 8080 et 8081 n'étaient publiés que dans le cluster, pas sur le PC.

## 2.3 Publier le vote et les résultats avec Traefik

Le service `result` reçoit les mêmes labels avec `result.swarm.localhost`, et les 2 services perdent leurs ports 8080 et 8081.

**voting-app/docker-stack.traefik.yml**

```
networks:
  - frontend
  - web
deploy:
  replicas: 2
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.vote.rule=Host(`vote.swarm.localhost`)"
    - "traefik.http.routers.vote.entrypoints=web"
    - "traefik.http.services.vote.loadbalancer.server.port=80"
```

En gardant le nom `voting`, Swarm met à jour les services existants au lieu d'en créer d'autres.

**Commandes**

```
docker compose cp ../../voting-app/docker-stack.traefik.yml manager:/home/manager/
docker compose exec manager docker stack deploy --with-registry-auth -c
↪  /home/manager/docker-stack.traefik.yml voting
docker compose exec manager docker stack services voting
```

**hosts**

```
127.0.0.1   vote.swarm.localhost
127.0.0.1   result.swarm.localhost
```

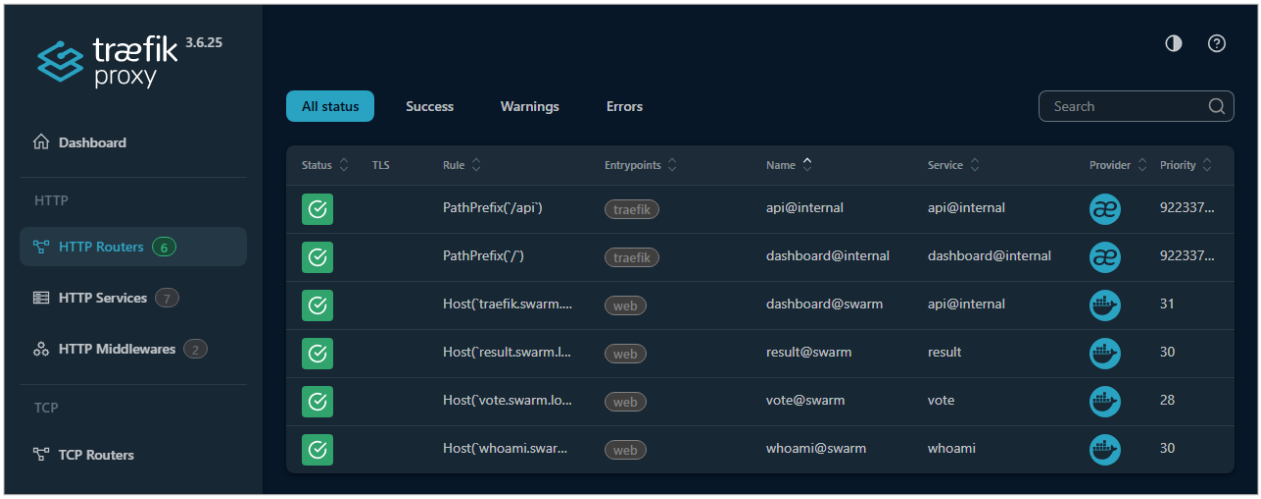



**Figure 3.5.** Les routeurs `vote@swarm` et `result@swarm` sont apparus tout seuls à partir des labels.

**Commandes**

```
curl -X POST -d vote=a http://vote.swarm.localhost/   # Cats, 2 fois
curl -X POST -d vote=b http://vote.swarm.localhost/   # Dogs, 3 fois
```

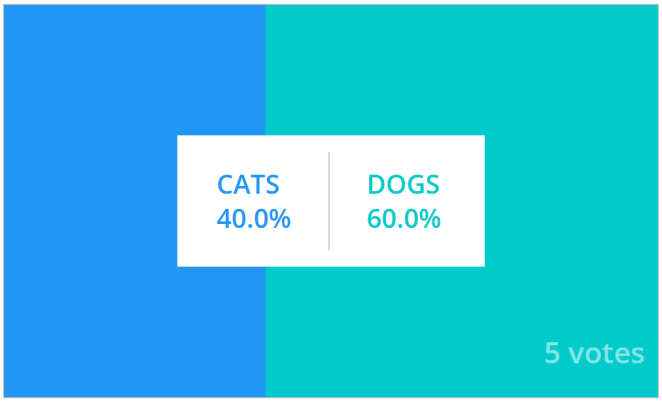



**Figure 3.6.** Le résultat est l'inverse du test local : la base du cluster ne contient que les votes envoyés au cluster.

# Exercice 3 : Déployer Portainer

## 3.1 Déployer Portainer sur le cluster

**Depuis mfp/swarm**

```
curl -sL https://downloads.portainer.io/ce-lts/portainer-agent-stack.yml -o
↪  portainer-agent-stack.yml
docker compose cp portainer-agent-stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy --with-registry-auth -c
↪  /home/manager/portainer-agent-stack.yml portainer
docker compose exec manager docker stack services portainer
docker compose exec manager docker ps --filter name=portainer --format 'table
↪  {{.Names}}\t{{.Image}}\t{{.Status}}'
```

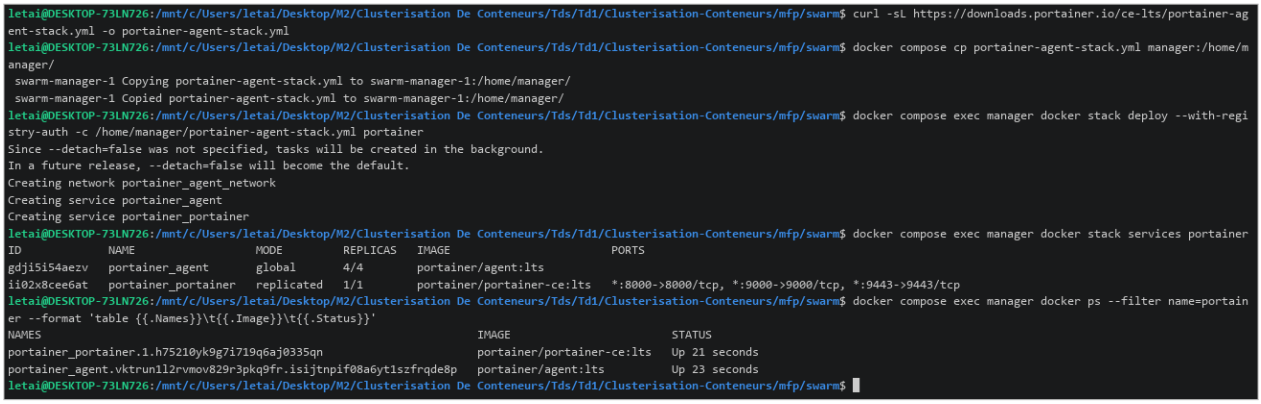



**Figure 3.7.** Un agent tourne sur chacun des 4 nœuds (mode `global`), et l'interface sur le manager.

## 3.2 Publier Portainer avec Traefik

**mfp/swarm/portainer-agent-stack.traefik.yml**

```yaml
networks:
  - agent_network
  - web
deploy:
  # ...
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.portainer.rule=Host(`portainer.swarm.localhost`)"
    - "traefik.http.routers.portainer.entrypoints=web"
    - "traefik.http.services.portainer.loadbalancer.server.port=9000"
```

**Commandes**

```
docker compose cp portainer-agent-stack.traefik.yml manager:/home/manager/
docker compose exec manager docker stack deploy --with-registry-auth -c
↪  /home/manager/portainer-agent-stack.traefik.yml portainer
docker compose exec manager docker stack services portainer
```

## 3.3 Ouvrir Portainer dans le navigateur

**hosts**

```
127.0.0.1   portainer.swarm.localhost
```

Portainer exige un setup token pour créer le compte administrateur : il l'écrit dans ses logs.

**Commandes**

```
docker compose exec manager docker service logs --raw portainer_portainer 2>&1 | grep -A2
↪  setup_token=
```

Le compte doit être créé dans les 5 minutes qui suivent le démarrage, sinon Portainer s'arrête par sécurité.

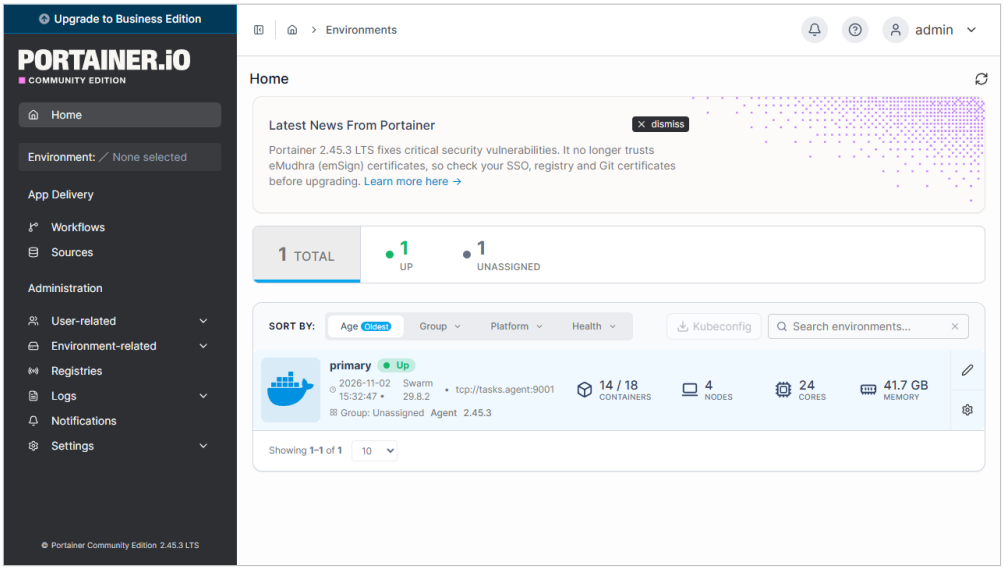



**Figure 3.8.** Portainer a trouvé le cluster grâce aux agents : un environnement Swarm avec ses 4 nœuds.

# Exercice 4 : Redéployer depuis Portainer

## 4.1 Supprimer la stack depuis le manager

**Depuis mfp/swarm**

```
docker compose exec manager docker stack ls
docker compose exec manager docker stack rm voting
docker compose exec manager docker stack ls
```

## 4.2 Republier la stack depuis Portainer

J'ai envoyé la version Traefik de la stack (question 2.3), pour que le vote et les résultats gardent leurs adresses.

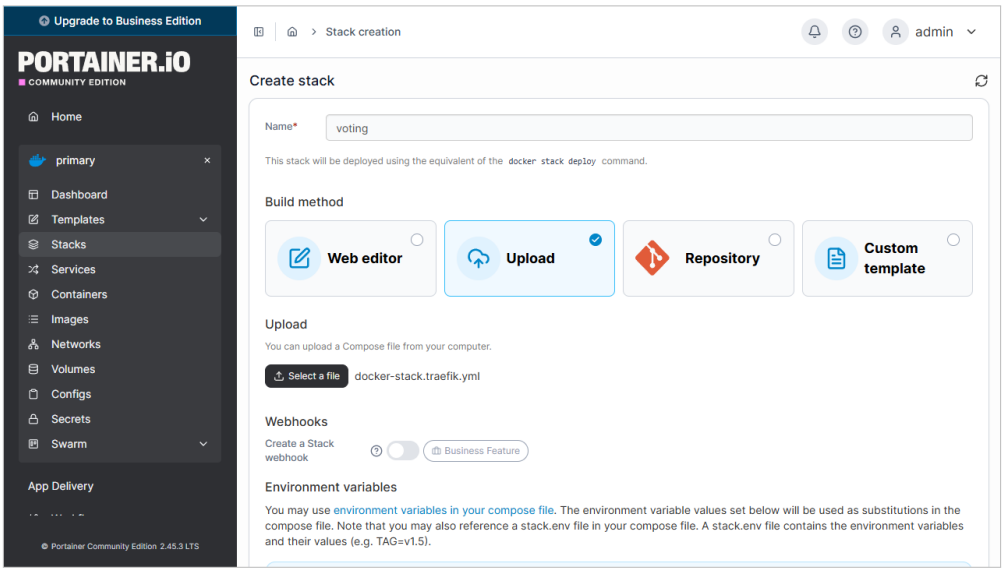



**Figure 3.9.** L'envoi du fichier dans Portainer fait l'équivalent d'un `docker stack deploy`.

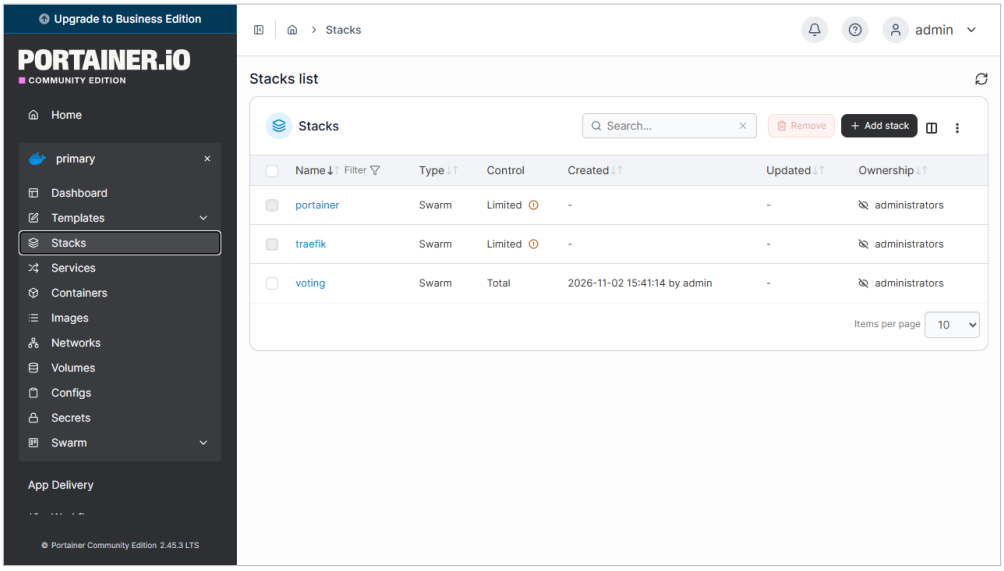



**Figure 3.10.** `voting` est en contrôle « Total », alors que les stacks créées en ligne de commande restent « Limited » : Portainer n'a pas leur fichier.

## 4.3 Vérifier l'accès depuis le navigateur

Le vote et les résultats répondaient de nouveau à leurs adresses.

> **Problème rencontré :** Les résultats affichaient « No votes yet » : la base a été recréée sur `node2`, avec un volume neuf.
>
> **Solution :** Rien à corriger : les anciens votes sont restés dans le volume de `node3`.

## 4.4 Déployer l'API de MFP depuis Portainer

### 4.4.a Création de la stack

Le serveur utilise l'image publiée sur GHCR, puisque Swarm ne construit pas d'images, et passe par Traefik au lieu de publier le port 3000.

**mfp/swarm/mfp-api.stack.yml**

```yaml
server:
  # ...
  deploy:
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.mfp-api.rule=Host(`api.swarm.localhost`)"
      - "traefik.http.routers.mfp-api.entrypoints=web"
      - "traefik.http.services.mfp-api.loadbalancer.server.port=3000"
```

Je l'ai déployée depuis Portainer sous le nom `mfp`, comme à la question 4.2.

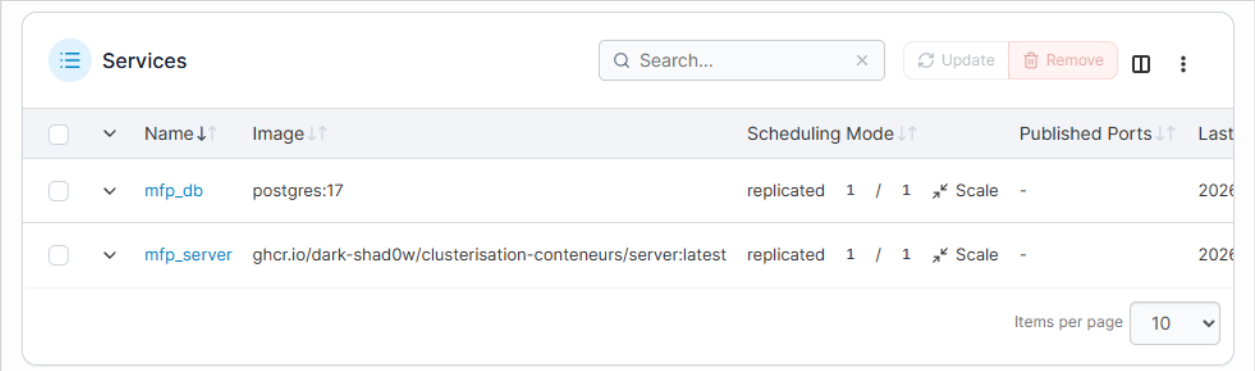



**Figure 3.11.** La base et le serveur de MFP tournent sur le cluster sans publier aucun port.

**Problème rencontré :** Au premier démarrage, le serveur s'arrêtait sur `getaddrinfo ENOTFOUND db` : la base n'existait pas encore.

**Solution :** Rien à faire : Swarm a relancé le serveur, qui s'est connecté au deuxième essai.

### 4.4.b Test avec Bruno

**hosts**

```
127.0.0.1   api.swarm.localhost
```

J'ai travaillé sur une copie de la collection Bruno, pour changer son adresse sans toucher à l'originale.

**Depuis mfp**

```
cp -r mfp-bruno-api-test-collection ~/mfp-bruno-swarm
sed -i 's#http://localhost:3000#http://api.swarm.localhost#' ~/mfp-bruno-swarm/*.yml
cd ~/mfp-bruno-swarm && bru run
```

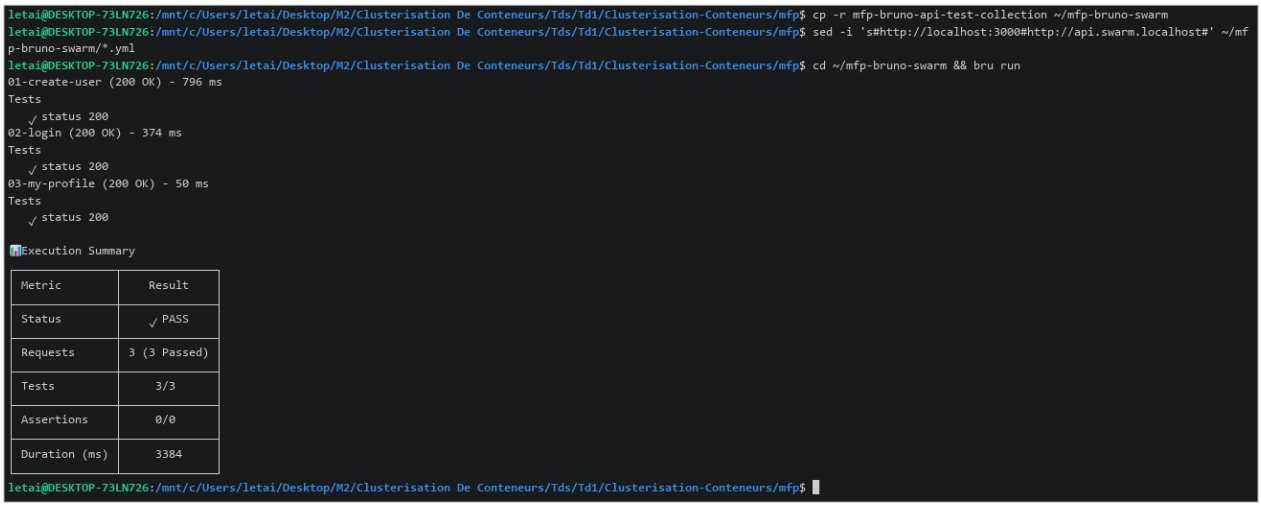



**Figure 3.12.** Les 3 requêtes passent : l'API du cluster répond depuis le PC sur `api.swarm.localhost`.

# TD 4 : Secrets et configs Docker Swarm

Swarm garde lui-même les secrets (informations sensibles, ni lisibles ni modifiables une fois créés) et les configs (fichiers lisibles mais non modifiables), et les monte comme des fichiers dans les conteneurs, quel que soit leur nœud.

## Prérequis : relancer le cluster et passer en HTTPS

### 0.1 Relancer le cluster

**swarm/compose.yml**

```yaml
manager:
  # ...
  ports:
    - "80:80" # Traefik, reachable from the host browser
    - "443:443" # Traefik HTTPS (TD4: Authelia only works over https)
```

**Commandes**

```
docker compose up -d
docker compose ps
docker compose exec manager docker node ls
```

**Problème rencontré :** Le manager, recréé pour le port 443, avait changé d'adresse IP : `node1`, `node2` et `node3` étaient `Down`.

**Solution :** J'ai fait quitter le swarm aux 4 nœuds, puis j'ai reconstruit le cluster.

**Commandes**

```
docker compose exec node1 docker swarm leave --force
docker compose exec node2 docker swarm leave --force
docker compose exec node3 docker swarm leave --force
docker compose exec manager docker swarm leave --force

docker compose exec manager docker swarm init
TOKEN=$(docker compose exec manager docker swarm join-token -q worker)
docker compose exec node1 docker swarm join --token $TOKEN manager:2377
docker compose exec node2 docker swarm join --token $TOKEN manager:2377
docker compose exec node3 docker swarm join --token $TOKEN manager:2377

docker compose exec manager docker node ls
```

Le réseau `web` et les stacks ont disparu avec l'ancien cluster ; les volumes, eux, sont restés sur les nœuds.

## 0.2 Créer le certificat

Authelia refuse une adresse en `http://`. J'ai donc créé une CA locale, ajoutée aux autorités de confiance de Windows, et un certificat `*.swarm.localhost` signé par elle.

**Commandes**

```
mkdir -p certs
openssl req -x509 -newkey rsa:2048 -noenc -days 3650 -subj "/CN=Swarm Local CA" -keyout
↪  certs/ca.key -out certs/ca.crt
openssl req -newkey rsa:2048 -noenc -subj "/CN=*.swarm.localhost" -keyout
↪  certs/swarm.localhost.key -out certs/swarm.localhost.csr
openssl x509 -req -in certs/swarm.localhost.csr -CA certs/ca.crt -CAkey certs/ca.key
↪  -CAcreateserial -days 365 -extfile <(printf
↪  "subjectAltName=DNS:*.swarm.localhost,DNS:swarm.localhost") -out
↪  certs/swarm.localhost.crt
ls certs
openssl verify -CAfile certs/ca.crt certs/swarm.localhost.crt
certutil.exe -user -addstore Root "$(wslpath -w certs/ca.crt)"
```




**Figure 4.1.** `openssl verify` répond `OK` : le certificat est signé par ma CA.

## 0.3 Le certificat dans le cluster : un secret et des configs

La clé privée est sensible et va dans un secret. Le certificat, public, et `traefik-tls.yml`, qui désigne ce certificat à Traefik, vont dans des configs.

**Commandes**

```
docker compose exec manager docker network create --driver overlay web
docker compose exec -T manager docker config create swarm_localhost_crt - <
↪  certs/swarm.localhost.crt
docker compose exec -T manager docker config create traefik_tls - < traefik-tls.yml
docker compose exec -T manager docker secret create swarm_localhost_key - <
↪  certs/swarm.localhost.key
docker compose exec manager docker config ls
docker compose exec manager docker secret ls
docker compose exec manager docker config inspect --pretty traefik_tls
docker compose exec manager docker secret inspect --pretty swarm_localhost_key
```

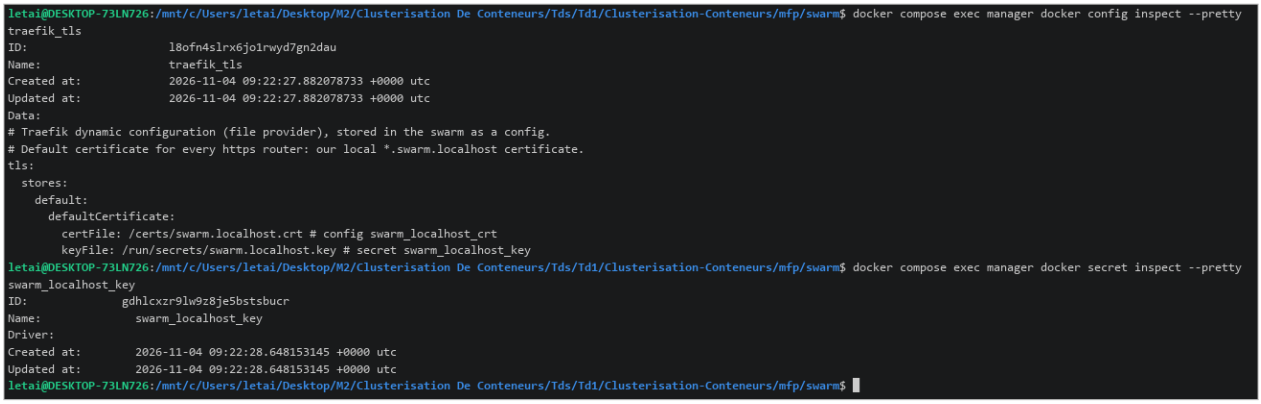



**Figure 4.2.** La config affiche son contenu (`Data`), le secret seulement son nom et ses dates.

## 0.4 Traefik en HTTPS

**swarm/traefik-https.stack.yml**

```yaml
networks:
  web:
    aliases:
      - authelia.swarm.localhost
configs:
  - source: traefik_tls
    target: /etc/traefik/tls.yml
  - source: swarm_localhost_crt
    target: /certs/swarm.localhost.crt
secrets:
  - source: swarm_localhost_key
    target: swarm.localhost.key # mounted in /run/secrets/
command:
  - "--entrypoints.websecure.address=:443"
  - "--entrypoints.websecure.http.tls=true"
```

**Commandes**

```
docker compose cp traefik-https.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/traefik-https.stack.yml
↪  traefik
docker compose exec manager docker service ls
curl -s --cacert certs/ca.crt https://whoami.swarm.localhost | head -4
curl -sI http://dozzle.swarm.localhost | head -2
```




**Figure 4.3.** `whoami` répond en HTTPS avec mon certificat, et une adresse http sans routeur est redirigée vers `https://`.

### 0.5 Relancer Portainer et ajouter les adresses

**Commandes**

```
docker compose exec manager docker volume ls
docker compose cp portainer-agent-stack.traefik.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c
↪  /home/manager/portainer-agent-stack.traefik.yml portainer
docker compose exec manager docker service ls
```

Le volume `portainer_portainer_data` est resté sur le manager : Portainer a retrouvé son compte `admin`.

**`hosts (Windows)`**

```
127.0.0.1 dozzle.swarm.localhost
127.0.0.1 authelia.swarm.localhost
127.0.0.1 outline.swarm.localhost
```

## Exercice 1 : Déployer des logiciels

### 1.1 Dozzle avec un utilisateur simple

#### 1.1.a Le fichier users.yml

**Commandes**

```
docker run --rm amir20/dozzle generate admin --password dozzle-admin-2026 --name "Admin"
```

#### 1.1.b Le secret, créé dans Portainer

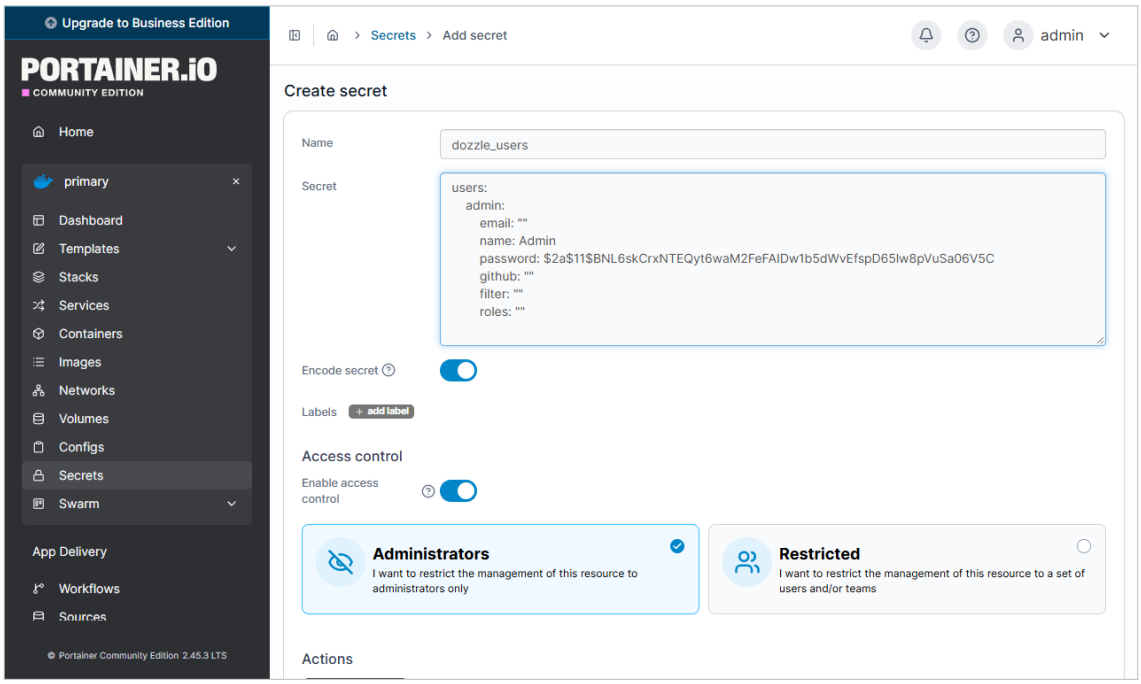



**Figure 4.4.** Le secret `dozzle_users` ne contient que l'empreinte bcrypt du mot de passe.

### 1.1.c La stack

**swarm/dozzle.stack.yml**

```
environment:
  DOZZLE_MODE: swarm # one Dozzle per node, they find each other on the "dozzle" network
  DOZZLE_AUTH_PROVIDER: simple # login form, users read from /data/users.yml
# ...
secrets:
  - source: dozzle_users
    target: /data/users.yml # where Dozzle looks for its users
```

**Commandes**

```
docker compose cp dozzle.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/dozzle.stack.yml dozzle
docker compose exec manager docker service ps dozzle_dozzle
```

### 1.1.d Test dans le navigateur

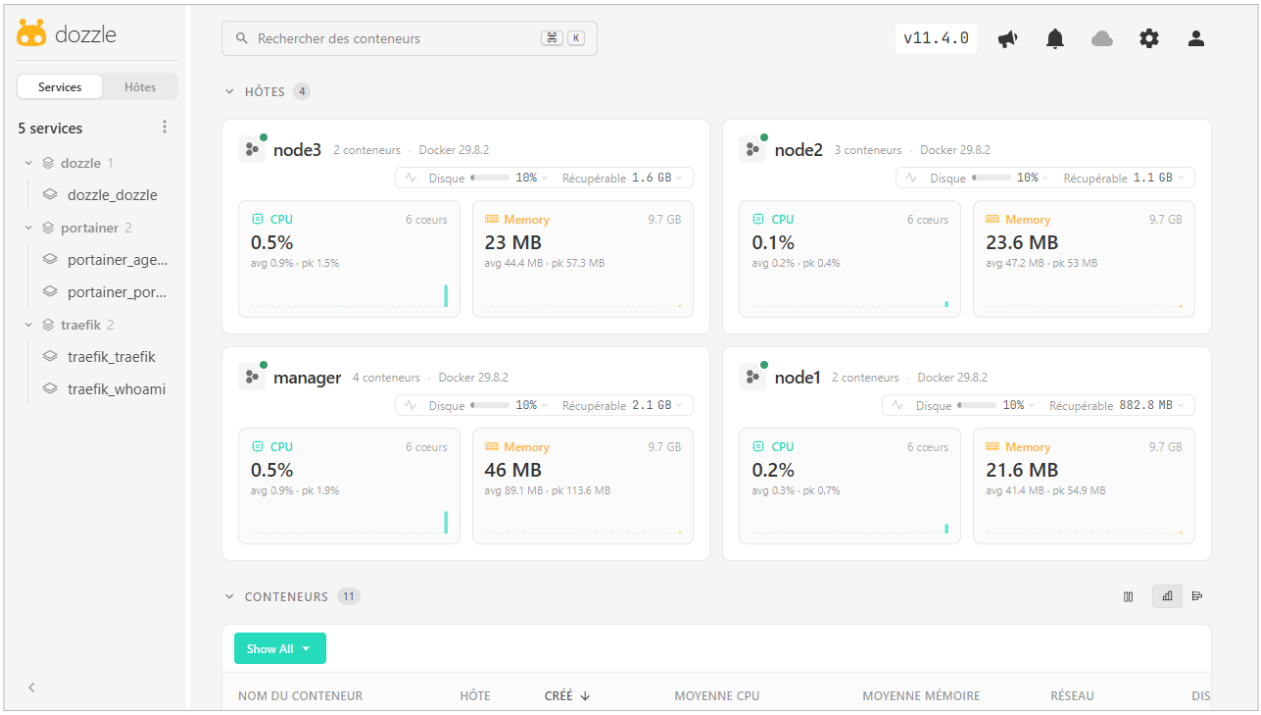



**Figure 4.5.** Connecté avec le compte `admin`, Dozzle montre les logs des 4 nœuds du cluster.

## 1.2 Authelia et connexion OpenID Connect

Avec OpenID Connect, Dozzle n'a plus de comptes : Authelia vérifie le mot de passe et lui renvoie un jeton signé.

### 1.2.a Les clés d'Authelia

Les clés passent directement d'`openssl` à Swarm, sans être écrites sur le disque.

**Commandes**

```
for s in authelia_jwt_secret authelia_session_secret authelia_storage_key
↪  authelia_oidc_hmac; do openssl rand -hex 32 | docker compose exec -T manager docker
↪  secret create $s -; done
openssl genrsa 2048 | docker compose exec -T manager docker secret create authelia_oidc_key
↪  -
docker compose exec -T manager docker config create swarm_ca - < certs/ca.crt
docker compose exec manager docker secret ls
```

### 1.2.b Le compte dans un secret

**Commandes**

```
docker run --rm authelia/authelia:4.39 authelia crypto hash generate argon2 --password
↪  'ahmed-authelia-2026'
cat secrets/users_database.yml
docker compose exec -T manager docker secret create authelia_users - <
↪  secrets/users_database.yml
```

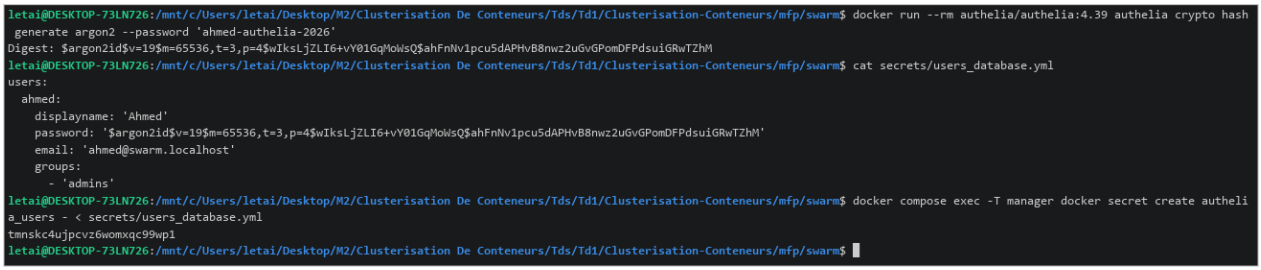



**Figure 4.6.** Le compte `ahmed`, du groupe `admins`, devient le secret `authelia_users`.

### 1.2.c Le client OIDC de Dozzle

Dozzle a besoin du client secret en clair, dans un secret. Authelia n'en garde que l'empreinte, dans sa configuration.

**Commandes**

```
openssl rand -hex 36 | tr -d "\n" > secrets/dozzle_oidc_secret
docker run --rm authelia/authelia:4.39 authelia crypto hash generate pbkdf2 --variant
↪  sha512 --password "$(cat secrets/dozzle_oidc_secret)"
docker compose exec -T manager docker secret create dozzle_oidc_secret - <
↪  secrets/dozzle_oidc_secret
```

### 1.2.d La configuration dans une config

La configuration d'Authelia ne contient aucune valeur secrète : elle peut donc aller dans une config.

**swarm/authelia.configuration.yml**

```
session:
  cookies:
    - domain: 'swarm.localhost'
      authelia_url: 'https://authelia.swarm.localhost'
# ...
```

```
    clients:
      - client_id: 'dozzle'
        client_secret: '$pbkdf2-sha512$310000$zQ.7CBa0mgiu8k8Zm3dTBA$cliYtETUut.EYaAb1C1x5G
        ↪ 0d8nJ7TBDq3ewmffU1bvgb/PbES0giHXdRx6Acqbgoa0V8yBz2Wk12VnIo7jdzLg'
        redirect_uris:
          - 'https://dozzle.swarm.localhost/api/auth/callback'
```

### 1.2.e Le déploiement

**swarm/authelia.stack.yml**

```
environment:
  AUTHELIA_SESSION_SECRET_FILE: /run/secrets/authelia_session_secret
  AUTHELIA_STORAGE_ENCRYPTION_KEY_FILE: /run/secrets/authelia_storage_key
  # ...
configs:
  - source: authelia_config
    target: /config/configuration.yml
secrets:
  - source: authelia_users
    target: /config/users_database.yml # the account(s)
  - authelia_session_secret
  # ...
```

**Commandes**

```
docker compose exec -T manager docker config create authelia_config - <
↪ authelia.configuration.yml
docker compose cp authelia.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/authelia.stack.yml authelia
docker compose exec manager docker service ls --filter name=authelia
docker compose exec manager docker service logs --raw authelia_authelia 2>&1 | tail -4
```

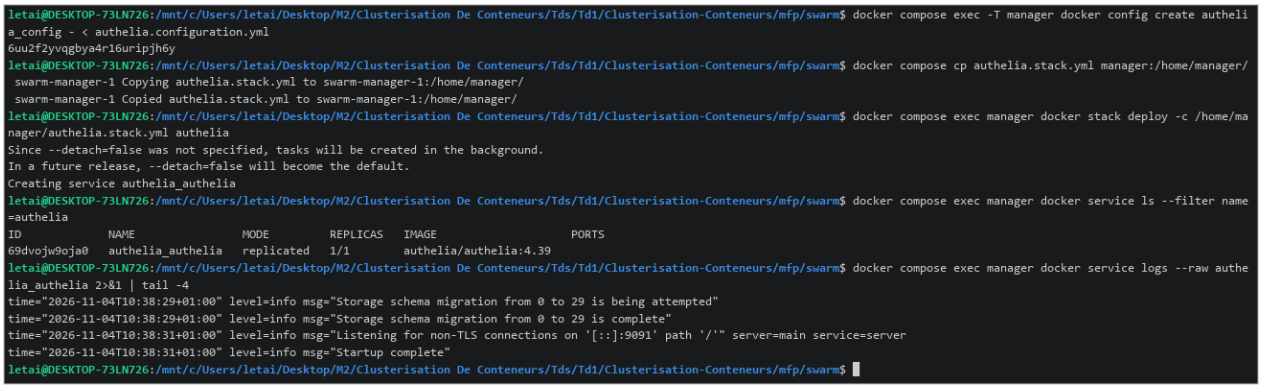



**Figure 4.7.** Authelia démarre avec sa config et ses secrets (`Startup complete`).

### 1.2.f Brancher Dozzle sur Authelia

Dozzle appelle lui-même `https://authelia.swarm.localhost` : l'alias de Traefik sur le réseau `web` lui donne ce nom, et la config `swarm_ca` lui fait accepter mon certificat.

**swarm/dozzle.oidc.stack.yml**

```
environment:
  DOZZLE_AUTH_PROVIDER: oidc # Authelia owns the accounts, users.yml is no longer read
  DOZZLE_AUTH_OIDC_ISSUER: https://authelia.swarm.localhost
  DOZZLE_AUTH_OIDC_CLIENT_ID: dozzle
  DOZZLE_AUTH_OIDC_CLIENT_SECRET_FILE: /run/secrets/dozzle_oidc_secret
  SSL_CERT_FILE: /certs/ca.crt
# ...
configs:
  - source: swarm_ca
    target: /certs/ca.crt
```

**Commandes**

```
docker compose cp dozzle.oidc.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/dozzle.oidc.stack.yml
↪  dozzle
docker compose exec manager docker service ps dozzle_dozzle --filter desired-state=running
docker compose exec manager docker service logs --raw --since 2m dozzle_dozzle 2>&1 | grep
↪  -iE 'oidc|auth|error|fatal' | head -8
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose cp dozzle.oidc.stack.yml manager:/home/manag
er/
 swarm-manager-1 Copying dozzle.oidc.stack.yml to swarm-manager-1:/home/manager/
 swarm-manager-1 Copied dozzle.oidc.stack.yml to swarm-manager-1:/home/manager/
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker stack deploy -c /home/ma
nager/dozzle.oidc.stack.yml dozzle
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Updating service dozzle_dozzle (id: fsw1thgvkzmjnuwpcn7aqlv4b)
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker service ps dozzle_dozzle
 --filter desired-state=running
ID             NAME                                IMAGE                NODE      DESIRED STATE   CURRENT STATE           ERROR     PORTS
3ajebiwrfo45   dozzle_dozzle.spjvxolb649yacz4jcv685jk3   amir20/dozzle:latest   node1     Running         Running 36 seconds ago
omrjaohv2wz2   dozzle_dozzle.w7ufj1jhxaf70ep1re4ouyllh   amir20/dozzle:latest   manager   Running         Running 42 seconds ago
bvdx7pdznxuk   dozzle_dozzle.wkc8d6w02b5j4ksw9jeh7l3bm   amir20/dozzle:latest   node2     Running         Running 30 seconds ago
m0qq5c5jmcta   dozzle_dozzle.yc475joofqmkybq45qayead8f   amir20/dozzle:latest   node3     Running         Running 20 seconds ago
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker service logs --raw --sin
ce 2m dozzle_dozzle 2>&1 | grep -iE 'oidc|auth|error|fatal' | head -8
{"level":"warn","version":"v11.4.0","file":"users.yml","time":"2026-11-04T09:40:05Z","message":"Ignoring the user database: with --auth-provider oidc every user comes from the token"}
{"level":"info","version":"v11.4.0","issuer":"https://authelia.swarm.localhost","rolesClaim":"dozzle_roles, resource_access.dozzle.roles, roles","time":"2026-11-04T09:40:05Z","message":
"Using OpenID Connect authentication; users and roles come from the token"}
{"level":"warn","version":"v11.4.0","file":"users.yml","time":"2026-11-04T09:39:55Z","message":"Ignoring the user database: with --auth-provider oidc every user comes from the token"}
{"level":"info","version":"v11.4.0","issuer":"https://authelia.swarm.localhost","rolesClaim":"dozzle_roles, resource_access.dozzle.roles, roles","time":"2026-11-04T09:39:55Z","message":
"Using OpenID Connect authentication; users and roles come from the token"}
{"level":"warn","version":"v11.4.0","file":"users.yml","time":"2026-11-04T09:39:49Z","message":"Ignoring the user database: with --auth-provider oidc every user comes from the token"}
{"level":"info","version":"v11.4.0","issuer":"https://authelia.swarm.localhost","rolesClaim":"dozzle_roles, resource_access.dozzle.roles, roles","time":"2026-11-04T09:39:49Z","message":
"Using OpenID Connect authentication; users and roles come from the token"}
{"level":"warn","version":"v11.4.0","file":"users.yml","time":"2026-11-04T09:39:43Z","message":"Ignoring the user database: with --auth-provider oidc every user comes from the token"}
{"level":"info","version":"v11.4.0","issuer":"https://authelia.swarm.localhost","rolesClaim":"dozzle_roles, resource_access.dozzle.roles, roles","time":"2026-11-04T09:39:43Z","message":
"Using OpenID Connect authentication; users and roles come from the token"}
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$
```

**Figure 4.8.** Les logs confirment le mode OpenID Connect, avec Authelia comme émetteur.

### 1.2.g Rôles manquants et mise à jour de la config

**Problème rencontré :** Au retour d'Authelia, Dozzle refusait la connexion : `no roles claim found in the ID token`.

**Solution :** Authelia calcule maintenant un attribut `dozzle_roles` à partir des groupes, que Dozzle demande avec `DOZZLE_AUTH_OIDC_SCOPES: dozzle_roles`.

**swarm/authelia.configuration.yml**

```
definitions:
  user_attributes:
    dozzle_roles:
      expression: '"admins" in groups ? ["all"] : []'
# ...
    claims_policies:
      dozzle:
        id_token: ['dozzle_roles']
        custom_claims:
          dozzle_roles: {}
    # ...
        claims_policy: 'dozzle'
```

Une config ne se modifie pas : je l'ai supprimée puis recréée.

**Problème rencontré :** Le premier `config rm` a été refusé, car la config était utilisée par le service `authelia_authelia`.

**Solution :** J'ai d'abord retiré la stack `authelia`, ce qui ne touche pas son volume.

**Commandes**

```
docker compose exec manager docker config rm authelia_config
docker compose exec manager docker stack rm authelia
docker compose exec manager docker config rm authelia_config
docker compose exec -T manager docker config create authelia_config - <
↪  authelia.configuration.yml
docker compose exec manager docker stack deploy -c /home/manager/authelia.stack.yml authelia
docker compose exec manager docker service ls --filter name=authelia
docker compose cp dozzle.oidc.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/dozzle.oidc.stack.yml
↪  dozzle
docker compose exec manager docker service ls --filter name=dozzle
```

### 1.2.h Résultat

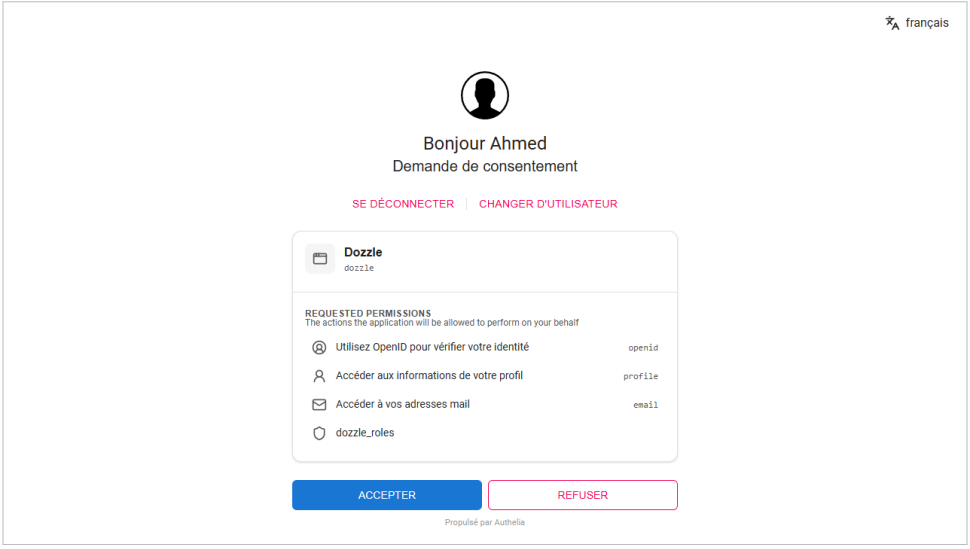



**Figure 4.9.** Le scope `dozzle_roles` fait maintenant partie de la demande.

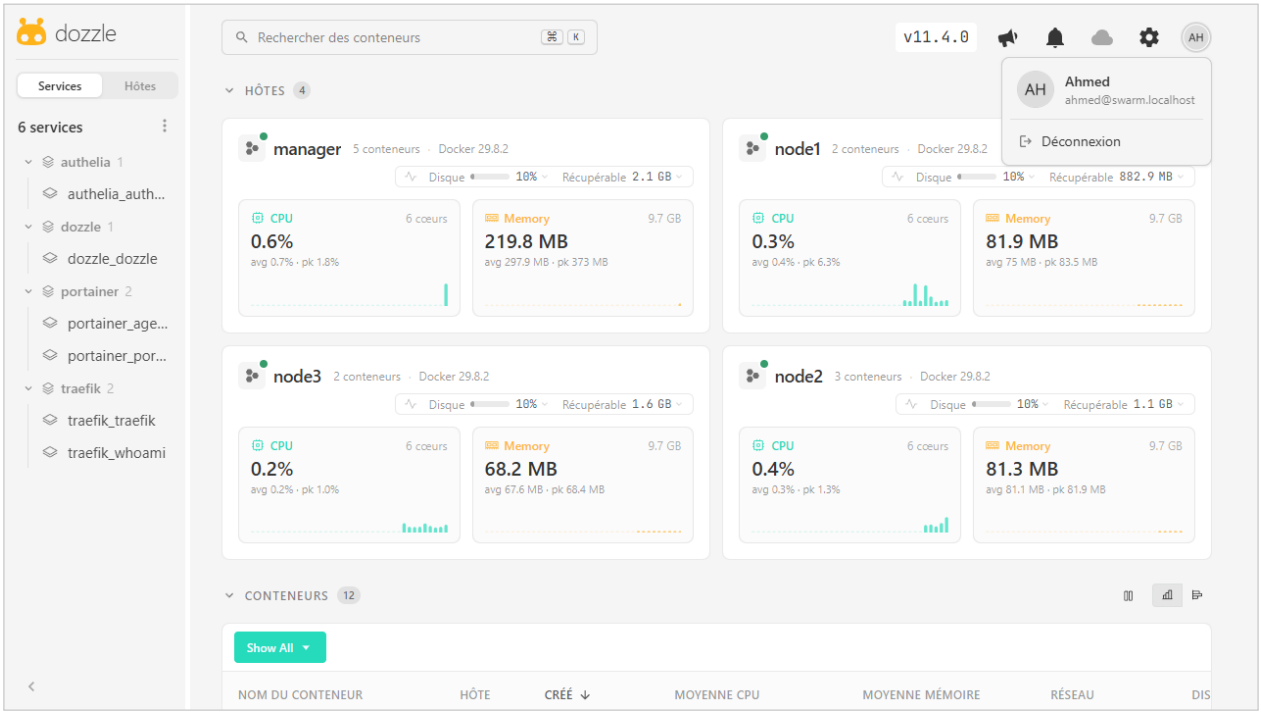



**Figure 4.10.** Connecté à Dozzle en tant qu'Ahmed, le compte du secret `authelia_users`.

## 1.3 Bonus : Outline

### 1.3.a Les secrets d'Outline

Le mot de passe de PostgreSQL, tiré une seule fois dans `PW`, sert à 2 secrets et n'est jamais écrit sur le disque.

**Commandes**

```
PW=$(openssl rand -hex 24)
printf %s "$PW" | docker compose exec -T manager docker secret create outline_db_password -
printf %s "postgres://outline:$PW@postgres:5432/outline" | docker compose exec -T manager
↪  docker secret create outline_database_url -
for s in outline_secret_key outline_utils_secret; do openssl rand -hex 32 | tr -d "\n" |
↪  docker compose exec -T manager docker secret create $s -; done
openssl rand -hex 36 | tr -d "\n" > secrets/outline_oidc_secret
docker run --rm authelia/authelia:4.39 authelia crypto hash generate pbkdf2 --variant
↪  sha512 --password "$(cat secrets/outline_oidc_secret)"
docker compose exec -T manager docker secret create outline_oidc_secret - <
↪  secrets/outline_oidc_secret
```

### 1.3.b Un nouveau client dans Authelia

J'ai ajouté un client `outline`, sur le modèle de celui de Dozzle. Comme le rappelle l'énoncé, la config a dû être supprimée puis recréée.

**Commandes**

```
docker compose exec manager docker stack rm authelia
docker compose exec manager docker config rm authelia_config
docker compose exec -T manager docker config create authelia_config - <
↪  authelia.configuration.yml
docker compose exec manager docker stack deploy -c /home/manager/authelia.stack.yml authelia
docker compose exec manager docker service ls --filter name=authelia
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker stack rm authelia
Removing service authelia_authelia
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker config rm authelia_confi
g
authelia_config
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec -T manager docker config create autheli
a_config - < authelia.configuration.yml
fmi0n8xr478ookq9vx611lfqn
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker stack deploy -c /home/ma
nager/authelia.stack.yml authelia
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating service authelia_authelia
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker service ls --filter name
=authelia
ID             NAME               MODE         REPLICAS   IMAGE                  PORTS
nbui8gga412a   authelia_authelia  replicated   1/1        authelia/authelia:4.39
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$
```

**Figure 4.11.** La stack étant retirée d'abord, `config rm` passe du premier coup.

### 1.3.c La stack

**swarm/outline.stack.yml**

```yaml
environment:
  SECRET_KEY_FILE: /run/secrets/outline_secret_key
  UTILS_SECRET_FILE: /run/secrets/outline_utils_secret
  DATABASE_URL_FILE: /run/secrets/outline_database_url
  # ...
  OIDC_CLIENT_ID: outline
  OIDC_CLIENT_SECRET_FILE: /run/secrets/outline_oidc_secret
  OIDC_AUTH_URI: https://authelia.swarm.localhost/api/oidc/authorization
  OIDC_TOKEN_URI: https://authelia.swarm.localhost/api/oidc/token
  OIDC_USERINFO_URI: https://authelia.swarm.localhost/api/oidc/userinfo
  # ...
  NODE_EXTRA_CA_CERTS: /certs/ca.crt
```

**Commandes**

```
docker compose cp outline.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/outline.stack.yml outline
docker compose exec manager docker stack services outline
docker compose exec manager docker service logs --raw outline_outline 2>&1 | tail -4
```

### 1.3.d Résultat

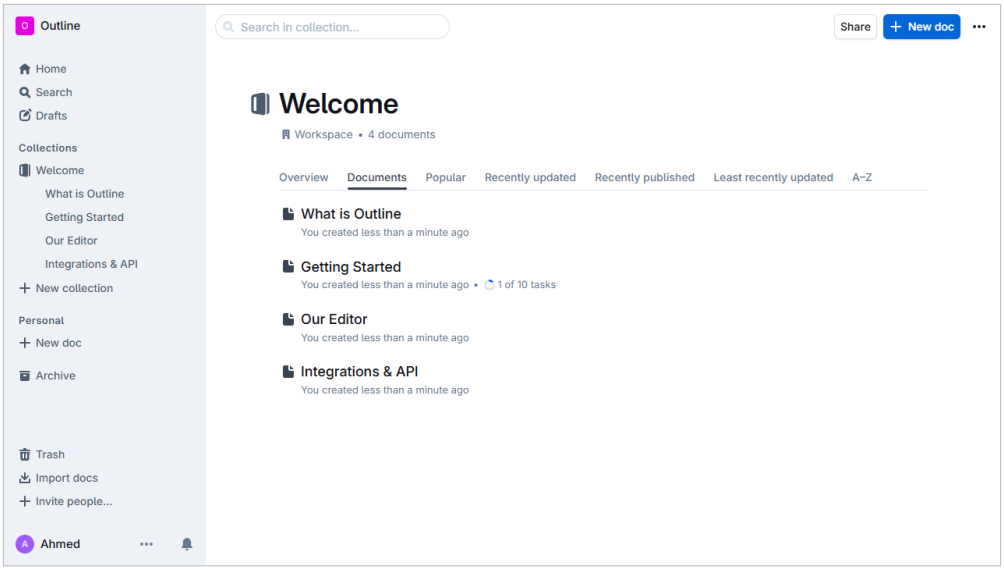



**Figure 4.12.** Connecté à Outline par Authelia, en tant qu'Ahmed.

## Bilan

| Nom | Type | Utilisé par | Créé depuis |
|---|---|---|---|
| `swarm_localhost_key` | secret | Traefik | ligne de commande |
| `swarm_localhost_crt`, `traefik_tls` | config | Traefik | ligne de commande |
| `swarm_ca` | config | Dozzle, Outline | ligne de commande |
| `dozzle_users` | secret | Dozzle (question 1.1) | Portainer |
| `dozzle_oidc_secret` | secret | Dozzle (question 1.2) | ligne de commande |
| `authelia_users` | secret | Authelia (le compte) | ligne de commande |
| `authelia_jwt_secret`, `authelia_session_secret`, `authelia_storage_key`, `authelia_oidc_hmac`, `authelia_oidc_key` | secret | Authelia | ligne de commande |
| `authelia_config` | config | Authelia | ligne de commande, recréée 2 fois |
| `outline_db_password` | secret | PostgreSQL d'Outline | ligne de commande |
| `outline_database_url`, `outline_secret_key`, `outline_utils_secret`, `outline_oidc_secret` | secret | Outline | ligne de commande |

Une config se relit, un secret jamais, et aucun des deux ne se modifie.

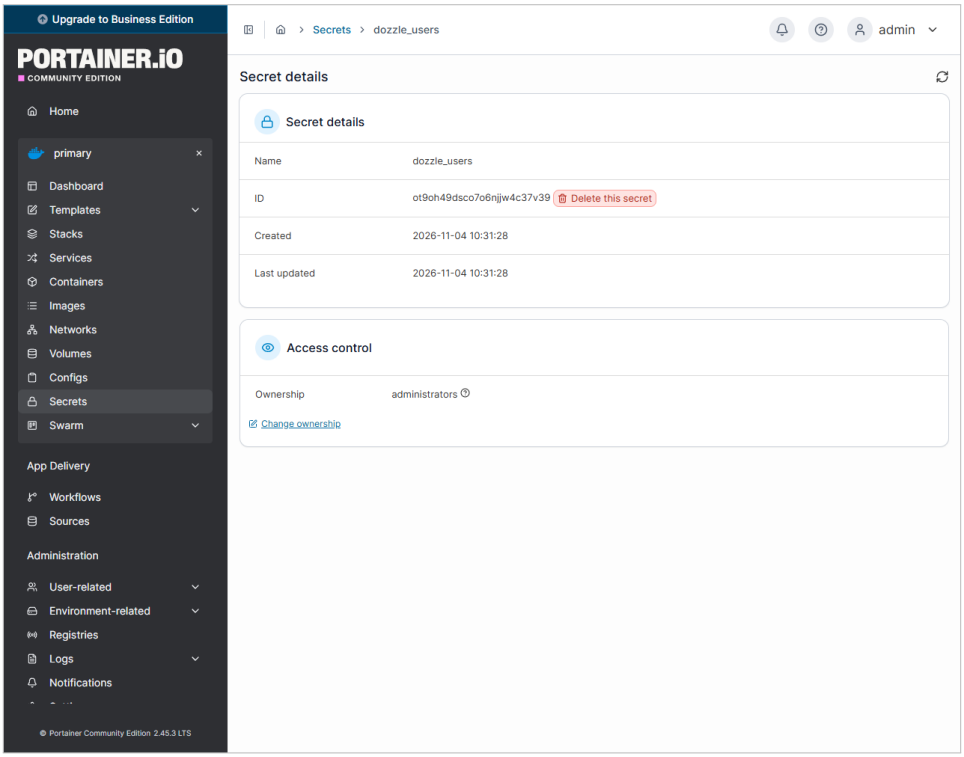



**Figure 4.13.** Le secret `dozzle_users` ne montre aucun contenu et ne peut qu'être supprimé.

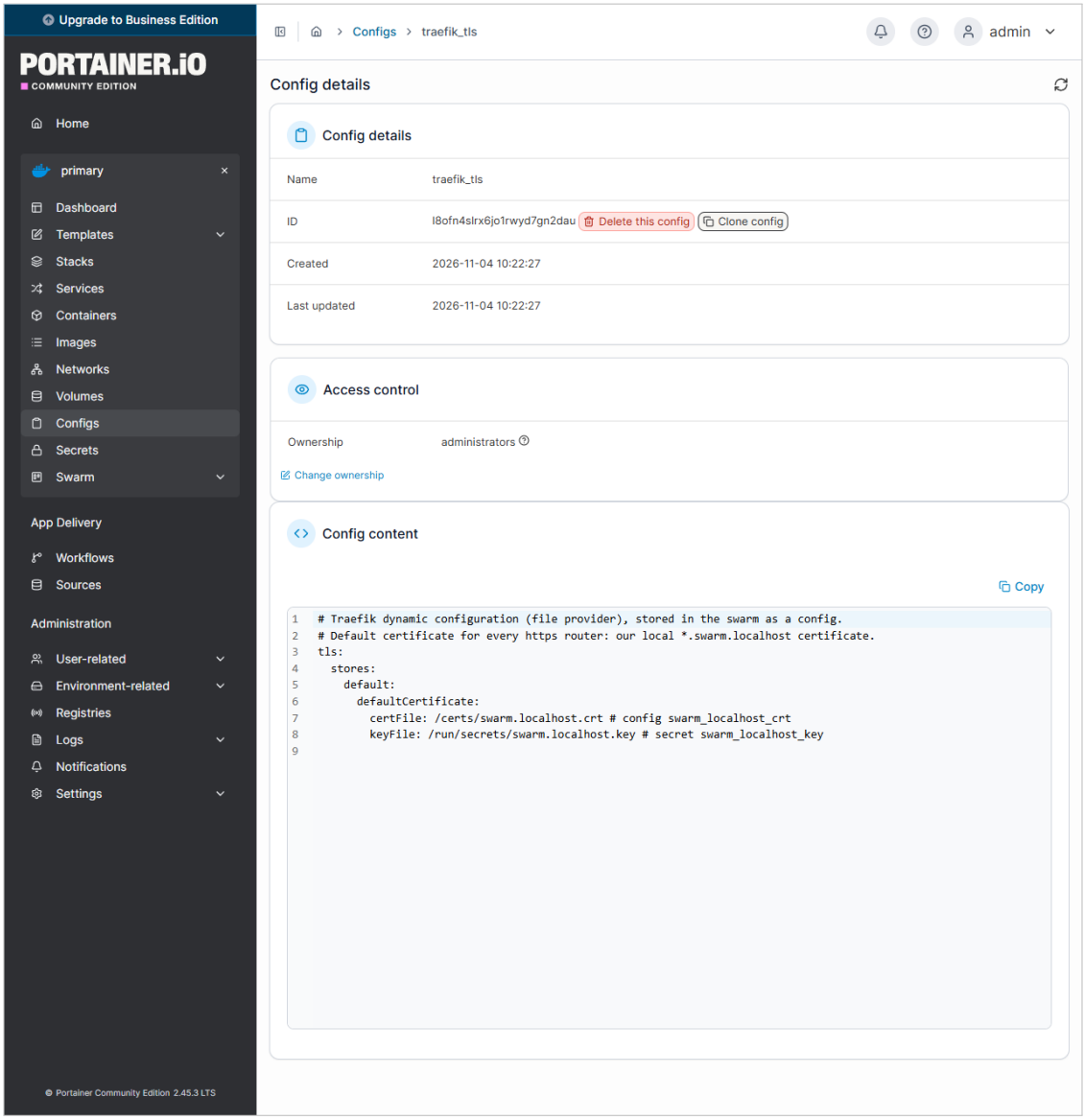



**Figure 4.14.** La config `traefik_tls` montre son contenu, mais ne peut qu'être supprimée ou clonée.